

Master's thesis

Master's Programme in Computer Science

Hiệu năng, an toàn và đánh đổi kỹ thuật của Rust và C++ trong hệ thống mặt điều khiển RAN của Nokia

Ha Vien

September 11, 2026

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Hiệu năng, an toàn và đánh đổi kỹ thuật của Rust và C++ trong hệ thống mặt điều khiển RAN của Nokia

Ohjaajat — Handledare — Supervisors

Prof. Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Master's thesis	September 11, 2026	41 pages

Tiivistelmä — Referat — Abstract

Bối cảnh. Phần mềm mặt điều khiển của mạng truy nhập vô tuyến (RAN) chủ yếu được viết bằng C++ vì hiệu năng, với cái giá đã biết về an toàn bộ nhớ và tương tranh. Rust hứa hẹn hiệu năng tương đương cùng an toàn tại thời điểm biên dịch, nhưng mức phù hợp cho môi trường nhạy độ trễ này vẫn chưa được chứng minh.

Mục tiêu. Luận văn so sánh C++ hiện đại và Rust trên một dịch vụ Quản lý Ngữ cảnh thiết bị người dùng (UE) mô hình hoá theo một thành phần mặt điều khiển trạm gốc 5G thực của Nokia (gNB-CU-CP). So sánh thông lượng trung bình để lại bốn chiều còn mở: độ trễ đuôi, những lớp lỗi C++ mà Rust ngăn khi biên dịch, chi phí mô hình an toàn, và chi phí áp dụng tăng dần.

Phương pháp. Một đặc tả độc lập ngôn ngữ được cố định trước, rồi hiện thực hai lần với thuật toán, mô hình tương tranh, bộ cấp phát và cấu hình biên dịch khớp nhau, và các khoảng cách bắt buộc do thành ngữ ngôn ngữ được ghi lại. Một khối lượng công việc tổng hợp được tăng đến quá tải, cả hai bản đều vượt cùng bộ kiểm thử tương đương, và phần đánh giá bổ sung tiêm lỗi cùng một nguyên mẫu giao diện hàm ngoại (FFI).

Kết quả. (1) Rust an toàn từ chối cả bảy lớp lỗi an toàn bộ nhớ và tranh chấp dữ liệu được tiêm vào tại thời điểm biên dịch. Ở C++ cùng bảy lớp đó biên dịch sạch. Chúng chỉ lộ ra lúc chạy, và chỉ khi bộ dò lỗi bắt được. (2) Bảy lớp nữa thất bại ở cả hai ngôn ngữ. Trên hai lớp trong số đó công cụ C++ báo cáo những gì Rust không báo. (3) C++ có thể gọi dịch vụ Rust qua một giao diện C mỏng. Toàn bộ mã `unsafe` nằm ở đó, và mọi thời gian thêm trên một cuộc gọi báo hiệu thực tối đa là 10.91% của cuộc gọi đó. (4) Một yêu cầu đầy đủ chỉ dành 0.250% cho công việc ngôn ngữ. Bàn giao giữa các luồng chiếm phần còn lại, nên đồng hồ đó không thể quy khoảng cách cho C++ hay Rust. Đo riêng, lõi nhanh hơn 1.6× ở Rust. (5) Dưới bão hoà, kiểm định ghép cặp trên trung vị đầu-cuối phân giải 1.3 μs nghiêng về Rust. Khoảng cách đó quá nhỏ để giới hạn một hiệu ứng ngôn ngữ. Các bản Rust theo kịp tải đưa vào đến 448 kilo-thao-tác mỗi giây (kops/s), các bản C++ chỉ đến 392. Khi cả hai đều không theo kịp, C++ vẫn hoàn thành 1.7× khối lượng của Rust trong 20/20 lần lặp đan xen với cả hai bộ cấp phát.

Kết luận. Chi phí chính khi áp dụng Rust là công sức kỹ thuật và đường cong học tập, không phải hiệu năng lúc chạy ở tải danh nghĩa. Lợi ích là thoát khỏi tranh chấp dữ liệu, chứ không phải mọi lỗi tương tranh. Một thử nghiệm ở tải danh nghĩa sẽ không lộ ra khoảng cách đó.

Avainsanat — Nyckelord — Keywords

Rust, C++, mặt điều khiển RAN, độ trễ đuôi, an toàn bộ nhớ, tiêm lỗi, áp dụng tăng dần

Säilytyspaikka — Förvaringsställe — Where deposited

Thư viện Đại học Helsinki

Muita tietoja — övriga uppgifter — Additional information

Software study track

ACM Computing Classification System (CCS)

Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Mục lục

Danh mục hình **iii**

Danh mục bảng **iv**

Danh mục từ viết tắt **v**

1 Giới thiệu **1**

1.1 Động lực 2
1.2 Bối cảnh công nghiệp 3
1.3 Phát biểu bài toán 4
1.4 Câu hỏi nghiên cứu và giả thuyết 5

2 Nền tảng: Ngôn ngữ hệ thống cho phần mềm nhạy độ trễ **7**

2.1 Hệ thống mặt điều khiển RAN và gNB-CU-CP 7
2.2 Khả năng dự đoán và hai ngôn ngữ 8
2.3 Khoảng trống nghiên cứu 8

3 Phương pháp nghiên cứu **11**

3.1 Phân hệ Quản lý Ngữ cảnh UE 11
3.2 Hướng tiếp cận nghiên cứu 12
3.3 Phương pháp theo từng câu hỏi 14
3.4 Tính công bằng như một biến được kiểm soát 14
3.5 Xử lý thống kê 15
3.6 Tái lập và hiện vật nghiên cứu 15

4 Thiết kế và hiện thực **16**

5 Thiết lập thí nghiệm và đánh giá **18**

5.1 Thiết lập hệ thống thử 18
5.2 Khối lượng và phép đo 19
5.3 Kết quả 20
5.3.1 Dưới bão hoà (RQ1, RQ3) 20

5.3.2 Qua bão hoà (RQ1) 21
5.3.3 Dùng tài nguyên (RQ3) 22
5.3.4 An toàn qua tiêm lỗi (RQ2) 23
5.3.5 Di trú (RQ4) 24

6 Thảo luận **26**
6.1 Tóm tắt phát hiện 26
6.2 Thảo luận các câu hỏi nghiên cứu 26
6.3 Xem lại các giả thuyết 28
6.4 Hướng dẫn kỹ thuật cho áp dụng tăng dần 29
6.5 Nơi nghiên cứu này đứng 29
6.6 Suy nghĩ đạo đức và nghề nghiệp 30

7 Kết luận và hướng phát triển **31**

Tuyên bố về việc dùng công cụ AI **33**

Tài liệu tham khảo **34**

Danh mục hình

2.1 Phân hệ nghiên cứu nằm ở đâu trong NG-RAN 5G 7
2.2 Các nghiên cứu lân cận kiểm soát gì, và để gì biến thiên 9

3.1 Máy trạng thái vòng đời ngữ cảnh UE 11

4.1 Kiến trúc không chia sẻ, một luồng mỗi phân mảnh 16

5.1 Hai khối lượng, và vòng đời mà khối mixed đi 19
5.2 Quét tải: độ trễ đuôi và thông lượng đạt được 21
5.3 Thao tác đầu-cuối được phân rã, đối với những gì quét phân giải được . 21
5.4 Phân kỳ qua bão hoà 22
5.5 Dùng tài nguyên, và hai hiệu ứng trong RSS đỉnh 23
5.6 Nơi bảo đảm dừng: ma trận kết cục 24
5.7 Đường nối di trú FFI từ C++ sang Rust 25

6.1 Nơi bảo đảm lúc biên dịch của Rust dừng 27

Danh mục bảng

3.1 Cách mỗi câu hỏi nghiên cứu được trả lời 13

5.1 Độ trễ mỗi thao tác tại tải đại diện 20

Danh mục từ viết tắt

Mỗi từ viết tắt dưới đây cũng được viết đầy đủ tại lần dùng đầu tiên trong văn bản.

Viết tắt	Dạng đầy đủ
3GPP	3rd Generation Partnership Project, cơ quan đặc tả các chuẩn 5G
ABI	Application binary interface, giao diện nhị phân ứng dụng
AI	Artificial intelligence, trí tuệ nhân tạo
AI-RAN	Kiến trúc trong đó khối lượng AI và mạng truy nhập vô tuyến dùng chung một hạ tầng gia tốc
API	Application programming interface, giao diện lập trình ứng dụng
ASN.1	Abstract Syntax Notation One
CPU	Central processing unit, bộ xử lý trung tâm
CVE	Common Vulnerabilities and Exposures, danh mục lỗ hổng phổ biến
CWE	Common Weakness Enumeration, danh mục điểm yếu phổ biến
DPDK	Data Plane Development Kit
EU	European Union, Liên minh châu Âu
FFI	Foreign function interface, giao diện hàm ngoại
FIFO	First in, first out, vào trước ra trước
FNV-1a	Hàm băm Fowler–Noll–Vo, biến thể 1a
GCC	GNU Compiler Collection
gNB	Next-generation NodeB, trạm gốc 5G
gNB-CU	Đơn vị trung tâm gNB
gNB-CU-CP	Mặt điều khiển đơn vị trung tâm gNB: lớp hệ thống được nghiên cứu ở đây
gNB-CU-UP	Mặt người dùng đơn vị trung tâm gNB
gNB-DU	Đơn vị phân tán gNB
HDR	High dynamic range, dải động cao, của histogram độ trễ
IMT-2030	International Mobile Telecommunications-2030, khung ITU cho 6G
ITU	International Telecommunication Union, Liên minh Viễn thông quốc tế
KPI	Key performance indicator, chỉ số hiệu năng then chốt
kops/s	Kilo-operations per second, kilo-thao-tác mỗi giây
KVM	Kernel-based Virtual Machine
MPSC	Multi-producer, single-consumer, nhiều tác nhân sinh, một tác nhân tiêu thụ
NG-RAN	Next-generation radio access network, mạng truy nhập vô tuyến thế hệ mới
PHY	Physical layer, tầng vật lý
RAII	Resource Acquisition Is Initialization
RAN	Radio access network, mạng truy nhập vô tuyến
RQ	Research question, câu hỏi nghiên cứu
RRC	Radio Resource Control, Điều khiển tài nguyên vô tuyến
RSS	Resident set size, kích thước tập trú
UE	User equipment, thiết bị người dùng như điện thoại hoặc modem
UECM	UE Context Management, phân hệ được tái hiện thực và đo ở đây

1 Giới thiệu

Mạng di động đang chuyển từ phần cứng chuyên dụng sang phần mềm trên máy chủ thông thường. Các chức năng từng chạy trên thiết bị tích hợp theo chiều dọc, chuyên biệt cố định, nay được tái hiện thực thành chương trình trên máy tính đa mục đích. Mạng truy nhập vô tuyến (RAN), lâu nay là phần phụ thuộc phần cứng nhất trong ngăn xếp, cũng nằm trong xu hướng đó.

Trong 5G, trạm gốc, tên chuẩn là next-generation NodeB (gNB), được tách thành một đơn vị trung tâm và một hoặc nhiều đơn vị phân tán. Đơn vị trung tâm lại được tách thành các chức năng mặt điều khiển và mặt người dùng [1, 2]. Open-RAN đẩy cùng ý tưởng xa hơn: RAN trở thành phần mềm ảo hoá, đa nhà cung cấp, nối với nhau qua giao diện mở [3]. Mặt điều khiển thu được, mặt điều khiển đơn vị trung tâm gNB (gNB-CU-CP), là một chương trình lớn, có trạng thái, tương tranh cao. Nó theo dõi mọi thiết bị đang kết nối, mỗi thiết bị mà các chuẩn gọi là thiết bị người dùng (UE), chạy báo hiệu Điều khiển tài nguyên vô tuyến (Radio Resource Control, RRC) [4], và điều phối di động cùng chuyển giao, tất cả trong giới hạn độ trễ nghiêm ngặt.

Tải tại một site bận rộn rất cao. Một site có thể chạy hàng nghìn thủ tục quản lý kết nối và di động mỗi giây, mỗi thủ tục phải hoàn thành trong ngân sách cỡ mili giây mà hệ thống 5G đặt cho báo hiệu mặt điều khiển [5]. Một mặt điều khiển triển khai có thể theo dõi hàng triệu thiết bị hoạt động cùng lúc. Khi RAN chuyển sang triển khai đám mây, tách rời, tính đúng đắn, độ tin cậy và thời gian của phần mềm này ngày càng quyết định chất lượng của toàn mạng. Một sự cố hoặc một đỉnh độ trễ đuôi ở đây không phải lỗi cục bộ. Mọi thiết bị site phục vụ đều bị ảnh hưởng [6, 7].

Phần mềm mặt điều khiển này, hiện nay, chủ yếu được viết bằng C++. Lựa chọn đó là hợp lý: C++ mang lại kiểm soát mức thấp, công cụ trưởng thành và hiệu năng mà hệ thống nhạy độ trễ đòi hỏi. Nhưng hiệu năng đó đi kèm chi phí đã được biết. C++ đặt gánh nặng quản lý bộ nhớ và tương tranh lên lập trình viên, và hàng thập kỷ kinh nghiệm cho thấy gánh nặng này thường xuyên bị xử lý sai, sinh ra lỗi use-after-free, double-free, tranh chấp dữ liệu và các lỗi liên quan, tốn kém để tìm, nguy hiểm khi vận hành, và chiếm phần lớn lỗ hổng bảo mật trong thực tiễn công nghiệp [8, 9]. Rust đã nổi lên như ngôn ngữ hệ thống đại chúng đầu tiên hứa hẹn thoát khỏi đánh đổi này: an toàn bộ nhớ và không tranh chấp dữ liệu được thực thi lúc biên dịch trên nền tảng đã được máy kiểm [10, 11], với hiệu năng dự kiến cạnh tranh với C++ [12]. Liệu lời hứa đó

có đúng với mặt điều khiển cấp viễn thông, nhạy độ trễ, nơi khả năng dự đoán khi quá tải quan trọng ngang tốc độ thuần, vẫn chưa được hiểu rõ. Luận văn này, thực hiện tại Nokia, trả lời câu hỏi trực tiếp: phát triển hai bản hiện thực tương đương về chức năng của một phân hệ RAN đại diện, một dịch vụ Quản lý Ngữ cảnh UE mô hình hoá theo một thành phần gNB-CU-CP thật của Nokia, bằng C++ hiện đại và Rust, rồi so sánh chúng dưới khối lượng báo hiệu tổng hợp sát thực tế.

Ba thuật ngữ lặp lại xuyên suốt. *gNB-CU-CP* là phần mềm mặt điều khiển mà công trình này xoay quanh. *Quản lý Ngữ cảnh UE (UECM)* là phân hệ cụ thể được tái hiện thực và đo lường. *Độ trễ đuôi*, các phân vị p99, p99.9 và p99.99 chứ không phải trung bình, là thuộc tính quyết định, hơn cả tốc độ trung bình, liệu hệ thống như vậy có đáp ứng hạn chót [6, 7].

1.1 Động lực

Ba xu hướng khiến phép so sánh này đúng thời điểm. Xu hướng thứ nhất là tiến hoá đám mây của RAN. Các chức năng mặt điều khiển nay chạy trên máy chủ đa mục đích, nên phần mềm sống lâu hơn, tương tranh cao hơn, và phải đáp ứng hạn chót mà không còn tính xác định của phần cứng chuyên dụng. Tách rời Open-RAN biến trạm gốc thành phần mềm ảo hoá, đa nhà cung cấp [3]. Một hồi quy độ trễ hoặc một lỗi hỏng bộ nhớ khi đó là rủi ro toàn mạng, và lựa chọn ngôn ngữ là một quyết định kỹ thuật.

Xu hướng thứ hai là trí tuệ nhân tạo (AI), vốn từng được mạng dùng để tự tinh chỉnh và nay được mạng mang như một khối lượng công việc riêng [13]. International Mobile Telecommunications-2030 (IMT-2030), khung 6G của Liên minh Viễn thông quốc tế (International Telecommunication Union, ITU), liệt kê vai trò thứ hai đó như một kịch bản sử dụng [14]. Kiến trúc hình thành quanh nó là AI-RAN, trong đó công việc AI và vô tuyến dùng chung một máy chủ gia tốc [15]. Mặt điều khiển ở đó không còn chiếm trọn máy chủ. Nó chia sẻ bộ xử lý trung tâm (CPU), bộ nhớ và cache với công việc suy luận mà nó không lập lịch. Khi hàng xóm đó chiếm dung lượng, mặt điều khiển bão hoà sớm hơn: nó ngừng theo kịp nhu cầu ở tải thấp hơn so với khi chạy một mình. Chương 5 tìm khoảng cách rộng nhất giữa hai ngôn ngữ sau điểm đó.

Xu hướng thứ ba là chuyển sang ngôn ngữ an toàn bộ nhớ. Các tổ chức phần mềm lớn báo cáo rằng phần lớn lỗ hổng bảo mật nghiêm trọng bắt nguồn từ lỗi an toàn bộ nhớ, và việc áp dụng ngôn ngữ an toàn bộ nhớ làm giảm chúng [8, 9, 16, 17]. Công trình độc lập lập luận rằng an toàn bộ nhớ nay cần yêu cầu được chuẩn hoá nếu muốn được mua sắm [18]. Các cơ quan an ninh mạng kêu gọi rời C và C++, và Đạo luật Cyber

Resilience của Liên minh châu Âu (EU) thêm nghĩa vụ pháp lý từ tháng 9 năm 2026 và tháng 12 năm 2027 [19, 20, 21]. Đạo luật không nêu tên ngôn ngữ lập trình nào. Bằng chứng trực tiếp cho một mặt điều khiển RAN vẫn còn thiếu. Cung cấp bằng chứng đó là động lực của công trình này.

1.2 Bối cảnh công nghiệp

Luận văn này được thực hiện tại Nokia. Công ty thuộc số ít nhà cung cấp xây, duy trì và phát triển phần mềm RAN sản xuất từ đầu đến cuối. Năm nhà cung cấp lớn nhất chiếm 96% doanh thu RAN toàn cầu trong ba quý đầu năm 2025, với mức tập trung theo chỉ số Herfindahl–Hirschman cao nhất trong mười năm [22, 23]. Phân hệ được nghiên cứu, UECM, được mô hình hoá theo một thành phần gNB-CU-CP thực chứ không phải ví dụ sách giáo khoa.

Trong kiến trúc của 3rd Generation Partnership Project (3GPP), gNB-CU-CP là nửa mặt điều khiển của đơn vị trung tâm. Nó lưu RRC và quản lý vòng đời ngữ cảnh UE [1, 4], dưới ngân sách cỡ mili giây mà hệ thống 5G đặt cho báo hiệu điều khiển [5]. Chương 2 đặt nó trong kiến trúc rộng hơn. Vì được viết bằng C++, nó thừa hưởng hiệu năng của ngôn ngữ đó cùng các mối nguy an toàn bộ nhớ và tương tranh. Ba xu hướng của Mục 1.1 gặp nhau ở đây. Phần mềm lớp này chạy trên mạng nhà khai thác đang vận hành. Lựa chọn ngôn ngữ là mối quan tâm sản xuất.

UECM đủ nhỏ để tái hiện thực trung thành và lý giải trọn vẹn. Nó vẫn mang những thuộc tính làm mã mặt điều khiển khó. Hiện thực bám theo một đặc tả công khai, đã cố định, không phải mã độc quyền, nên độc giả ngoài Nokia có thể tái lập phép so sánh.

Một quyết định có tầm một thế hệ. Một lựa chọn ngôn ngữ trong phần mềm lớp này sống lâu hơn thế hệ mà nó được đưa ra. 3GPP đã dành Release 20 cho các nghiên cứu 6G và Release 21 cho công việc chuẩn tắc. ITU yêu cầu đề xuất công nghệ vào đầu 2029 và nộp đặc tả vào giữa 2030 [24]. Công việc đó bao gồm di trú từ 5G và liên vận hành với 5G. Một mặt điều khiển viết hôm nay được duy trì cùng thế hệ kế, chứ không bị loại. IMT-2030 vẫn nêu an ninh và khả năng phục hồi như nguyên tắc thiết kế và vẫn mang kịch bản độ trễ thấp siêu tin cậy ở dạng được tăng cường [14].

Các phép đo ở đây dùng một phân hệ thế hệ hiện tại và các yêu cầu đã công bố. Không tuyên bố nào được đưa ra về những yêu cầu chưa được viết.

Một dạng lỗi cụ thể. Hãy xét một cơn bão di động, nhiều UE chuyển giao cùng lúc. Một ngữ cảnh UE được giải phóng khi nhả, trong khi một thông điệp báo hiệu đang bay vẫn còn tham chiếu tới nó, là lỗi use-after-free. Ở C++ điều đó biên dịch và có thể vượt các kiểm thử thông thường. Nó rồi có thể làm hỏng trạng thái của một UE khác hoặc làm sập luồng xử lý dưới tương tranh cao. Cùng lỗi đó có thể hiện ra như một đỉnh độ trễ đuôi thay vì một sự cố. Ở Rust an toàn, thiết kế không biên dịch: sở hữu và mượn từ chối tham chiếu treo. Đó là lý do luận văn này đo cả *an toàn* (những lỗi nào được ngăn, và khi nào) lẫn *khả năng dự đoán* (độ trễ đuôi khi quá tải), chứ không chỉ thông lượng trung bình.

1.3 Phát biểu bài toán

Hai thuộc tính của miền này làm lựa chọn ngôn ngữ mang hệ quả. Thứ nhất, quản lý bộ nhớ và tương tranh thủ công trong C++ phơi phần mềm trước các lỗi như use-after-free, double-free, tranh chấp dữ liệu và vô hiệu hoá iterator, im lặng lúc biên dịch và tốn kém để tìm và sửa ở quy mô viễn thông. Thứ hai, cái quyết định liệu hạn chót RAN có được đáp ứng không phải thông lượng trung bình mà là hành vi trường hợp xấu nhất và đuôi dưới tải và quá tải [6], một thuộc tính dễ hồi quy và khó đo đúng. Rust hứa loại lớp lỗi thứ nhất tại thời điểm biên dịch trong khi tuyên bố hiệu năng cạnh tranh trên lớp thứ hai, nhưng tuyên bố đó chủ yếu dựa trên các benchmark đa mục đích chứ không trên khối lượng viễn thông có trạng thái, tương tranh, nhạy độ trễ.

Gộp các mạch đó lại, bài toán cụ thể mà luận văn giải quyết như sau. Các nhà cung cấp viễn thông đang chịu áp lực ngày càng lớn, vừa kỹ thuật vừa ngày càng mang tính điều tiết, phải áp dụng ngôn ngữ an toàn bộ nhớ, nhưng với mặt điều khiển RAN họ thiếu bằng chứng *trực tiếp, có kiểm soát* để dựa vào quyết định đó. Chưa được xác lập liệu một hiện thực Rust của một phân hệ mặt điều khiển có trạng thái, tương tranh có thể khớp C++ hiện đại về *độ trễ đuôi và khả năng dự đoán khi quá tải* (khác với thông lượng trung bình), liệu các bảo đảm lúc biên dịch của Rust có thực sự loại những lớp lỗi cụ thể tác động tới mã C++ sẵn có, và chi phí áp dụng tăng dần *tốn* bao nhiêu bên trong một mã nguồn C++ lớn. Bằng chứng sẵn có phần lớn là đa mục đích và trường hợp trung bình [12, 25], để lại ba câu hỏi này (khả năng dự đoán, phạm vi lỗi được ngăn, và chi phí di trú) còn mở cho một phân hệ đại diện. Trả lời chúng trong điều kiện có kiểm soát, tái lập được là bài toán luận văn này đặt ra để giải.

1.4 Câu hỏi nghiên cứu và giả thuyết

Luận văn được tổ chức quanh bốn câu hỏi nghiên cứu. Chúng đặt khả năng dự đoán độ trễ lên trước, rồi phạm vi lỗi, rồi chi phí tài nguyên, rồi áp dụng tăng dần:

RQ1 Rust và C++ hiện đại so sánh thế nào về khả năng dự đoán độ trễ (trung vị, độ trễ đuôi p99, p99.9 và p99.99, và cực đại) cho một khối lượng mặt điều khiển RAN dưới tải tăng dần và quá tải?

RQ2 Những lớp lỗi an toàn bộ nhớ và tương tranh xảy ra trong C++ nào bị loại tại thời điểm biên dịch trong hiện thực Rust, và các lớp còn lại được phát hiện ở giai đoạn nào với chi phí nào?

RQ3 Chi phí, nếu có, của mô hình an toàn Rust (kiểm biên, bản sao do sở hữu, chi phí trừu tượng) trên thông lượng, mức dùng CPU, và hành vi bộ nhớ là bao nhiêu?

RQ4 Những hệ quả thực tiễn (công sức, khả năng tương tác qua giao diện hàm ngoại (FFI), và rủi ro) của việc áp dụng Rust tăng dần trong một hệ sinh thái phần mềm RAN C++ sẵn có là gì?

Ba giả thuyết được viết ra trước mọi phép đo.

H1 Với khối lượng này hai ngôn ngữ tương đương về hiệu năng dưới bão hoà, mọi khác biệt nếu có chỉ nằm ở đuôi xa.

H2 Rust an toàn ngăn tại thời điểm biên dịch những lớp lỗi an toàn bộ nhớ và tương tranh mà C++ chỉ cố bắt sau khi chương trình đã được xây.

H3 Thời gian thêm khi C++ gọi sang Rust nhỏ so với chi phí của chính các thao tác, nên chi phí thực của việc thêm Rust từng phần là công việc kỹ thuật, không phải tốc độ lúc chạy.

Chương 6 xem lại cả ba dựa trên bằng chứng.

Danh mục lỗi cho RQ2 gồm các lớp mà Rust an toàn được kỳ vọng ngăn và các lớp mà nó được kỳ vọng không ngăn. Nghiên cứu khi đó có thể tìm nơi bảo đảm an toàn dừng lại, thay vì chỉ cho thấy bảo đảm đó giữ.

Một nghiên cứu trước đã ước lượng biên đó từ một danh mục, không từ các chương trình được biên dịch. Nó lấy Common Weakness Enumeration (CWE), một danh sách dài các kiểu lỗi, và đánh dấu mỗi kiểu là kiểu mà Rust an toàn sẽ ngăn hoặc sẽ không.

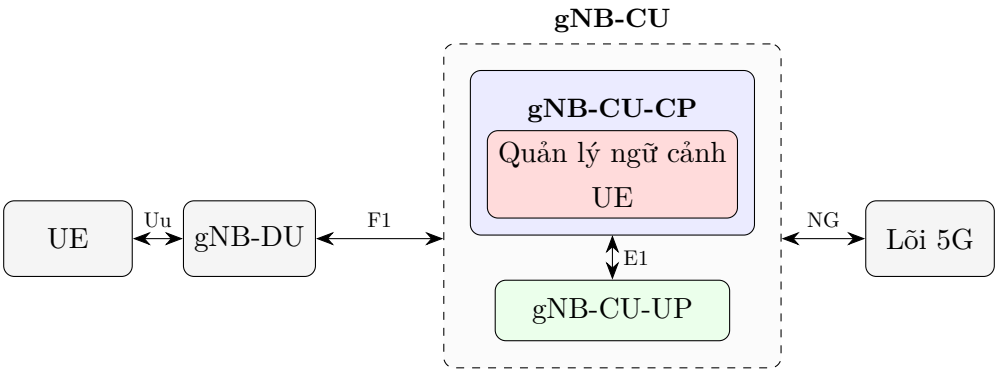
Nó rồi đếm các lỗ hổng thực trong 68 dự án C nguồn mở. 58.2% các lỗ hổng được phân loại nằm ở những kiểu mà Rust an toàn gần như đã làm không thể xảy ra [26]. Tỉ lệ đó là trường hợp tốt nhất: nó giả định mã mới ở lại trong Rust an toàn. Một kiểu được tính là đã ngăn nếu ánh xạ nói vậy, dù không trình biên dịch nào đã từ chối một chương trình. RQ2 hỏi cùng câu hỏi đó với các chương trình được biên dịch và chạy. Đó là cách nó có thể nói khi nào các kiểu còn lại bị bắt, và tốn gì để bắt chúng.

2 Nền tảng: Ngôn ngữ hệ thống cho phần mềm nhạy độ trễ

2.1 Hệ thống mặt điều khiển RAN và gNB-CU-CP

Mạng truy nhập vô tuyến thế hệ mới (NG-RAN) là bối cảnh của phép so sánh này. Tốc độ sự kiện cao: UE di chuyển, và báo hiệu có thể tăng đột biến. Giới hạn trễ chặt và phải ổn định. Các đặc tả dịch vụ và RAN 5G nêu những giới hạn đó như chỉ số hiệu năng then chốt (KPI) [5, 27]. Phần mềm sống lâu và tiếp tục lớn. Nhiều luồng chạy cùng lúc. Ở đây trường hợp xấu nhất và đuôi quan trọng hơn trung bình. Đó là lý do luận văn này đo đuôi.

gNB được tách theo hai trục. Tách thứ nhất là trung tâm đối với phân tán. Một đơn vị trung tâm (gNB-CU) lưu các tầng cao hơn của ngăn xếp. Một hoặc nhiều đơn vị phân tán (gNB-DU) lưu các tầng thấp hơn. Những tầng đó nằm gần vô tuyến và chặt về thời gian. Tách thứ hai là mặt điều khiển đối với mặt người dùng, bên trong đơn vị trung tâm. Nó tách chức năng mặt điều khiển (gNB-CU-CP) khỏi chức năng mặt người dùng (gNB-CU-UP). Các phần này nói chuyện qua liên kết chuẩn. NG đi về lõi 5G. F1 nằm giữa CU và DU. E1 nằm giữa CU-CP và CU-UP [1, 2, 28]. Open-RAN đẩy cùng các liên kết đó xa hơn. Mỗi phần khi đó có thể đến từ một nhà cung cấp khác và chạy như phần mềm trên máy thông thường [3]. Hình 2.1 đặt phân hệ được nghiên cứu ở đây, Quản lý Ngữ cảnh UE (UECM), bên trong gNB-CU-CP. UECM sở hữu trạng thái mặt điều khiển theo từng UE. Nó cũng xử lý báo hiệu RRC và di động. Đó là lý do tính đúng đắn và thời gian của nó đứng cho mặt điều khiển nói chung.



Hình 2.1: Phân hệ nghiên cứu nằm ở đâu trong NG-RAN 5G. Quản lý Ngữ cảnh UE được tô đậm bên trong gNB-CU-CP.

2.2 Khả năng dự đoán và hai ngôn ngữ

Mục này nêu những gì nghiên cứu lấy làm cho trước. Độ trễ ổn định, không phải thông lượng trung bình, là thuộc tính chính của một dịch vụ nhạy trễ. Vì vậy đại lượng cần đo là đuôi, không phải trung bình. Đo đuôi tốt cũng định hình khung đo. Lịch gửi phải là vòng hở. Một bộ gửi vòng kín dừng khi một tắc đang diễn ra. Nó rồi không bao giờ ghi lại độ trễ của tắc đó.

C++ cho kiểm soát mà miền cần. Nó để an toàn bộ nhớ và luồng cho lập trình viên. Bộ dò lỗi và rà soát đứng thay cho một bảo đảm. Công việc của uỷ ban chuẩn về hồ sơ an toàn được thực thi chưa phải một chuẩn. Nó chưa được dùng rộng. Sở hữu và mượn của Rust loại lỗi vòng đời và aliasing tại thời điểm biên dịch. Các trait `Send` và `Sync` loại tranh chấp dữ liệu. Tuyên bố đó hẹp hơn "an toàn luồng". Nó không nói gì về thứ tự khoá. Nó không nói gì về việc một bước ghép có nguyên tử hay không. Nó không nói gì về rò rỉ. Và nó tắt ở mọi nơi chương trình viết `unsafe`. Cả hai ngôn ngữ đều diễn đạt được cùng cấu trúc luồng. Một phép so sánh để cấu trúc đổi theo ngôn ngữ khi đó đo cấu trúc, không đo ngôn ngữ. Hai bên gặp nhau qua một giao diện nhị phân ứng dụng C. Đó là đường mà mọi di trú từng bước phải dùng. Đó cũng là lý do một đường nối giữ `unsafe` mà một cuộc di trú cần.

2.3 Khoảng trống nghiên cứu

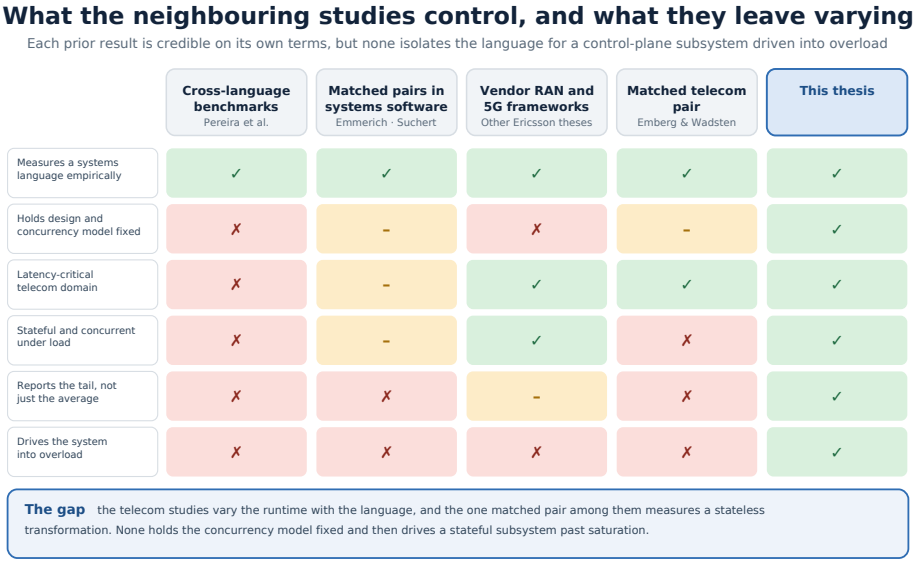
Các mục trên dựng khoảng trống. Mặt điều khiển RAN có nhiều luồng và giới hạn trễ chặt. Độ trễ đuôi khi tải quá lớn, không phải thông lượng trung bình, đặt chất lượng mà mạng giao. C++ cho kiểm soát và tốc độ mà miền cần. Nhưng nó để cả lớp lỗi bộ nhớ và luồng cho phát hiện sau. Rust hứa chặn những lớp đó tại thời điểm biên dịch mà không tốn thời gian chạy. Nó cũng cho một đường áp dụng từng bước qua giao diện nhị phân ứng dụng (ABI) C.

Miền này không phải chưa được nghiên cứu. Khối lượng công trình trước dễ bị bỏ sót, vì gần như không công trình nào được lập chỉ mục. Năm luận văn thạc sĩ do nhà cung cấp lưu trữ, tất cả thực hiện tại Ericsson, đã so sánh các ngôn ngữ ứng viên cho phần mềm RAN và viễn thông. Một bài cân chi phí lúc chạy với tốc độ con người xây được, trên C++, Rust và Go [29]. Một bài hỏi liệu Rust có thể thay C trong quản lý nút RAN [30]. Một bài xây bộ xử lý gói, một luồng mỗi nhân, bằng Rust async. Nó đo bộ xử lý đó đối với chương trình Data Plane Development Kit (DPDK) mà nó nhái [31]. Một bài đo độ trễ trung vị và p99 cho các bước mặt điều khiển trên năm khung async [32]. Bài cuối

xây các bản C++ và Rust khớp nhau của một bộ dịch ngược Abstract Syntax Notation One (ASN.1). Nó so sánh chúng bằng kiểm định thống kê [33]. Cả năm đều liên quan trực tiếp. Hai bài báo cáo khoảng cách không nghiêng về Rust.

Ba trong số đó đổi ngôn ngữ cùng lúc với thời gian chạy và mô hình luồng. Một khoảng cách đo được khi đó không thể quy cho ngôn ngữ chứ không phải khung [29, 31, 32]. Cỡ hiệu ứng lên tới hệ số năm. Điều đó cho thấy thời gian chạy chiếm bao nhiêu của phép đo. Hai bài kia giữ thiết kế sát hơn với cố định. Nhưng chúng báo cáo thời gian chạy và bộ nhớ đỉnh, không phải độ trễ đuôi dưới tải tăng [30, 33]. Bài gần nhất, Emberg và Wadsten, đo một phép biến đổi không trạng thái. Nó không phải một phần nhiều luồng có trạng thái. Hình 2.2 đặt năm luận văn nhà cung cấp cạnh các so sánh đa mục đích [34] và các cặp phần mềm hệ thống khớp [35, 36]. Mẫu hình như sau. Mỗi nhóm giữ một phần những gì một phép thử công bằng cần. Không nhóm nào giữ hết.

Sự tập trung đó tự nó là một phần của khoảng trống. Cả năm nghiên cứu đến từ một nhà cung cấp. Kết luận của chúng chưa bao giờ được thử đối với phần mềm, mục tiêu hay quy tắc nội bộ của một công ty khác. Vì vậy những gì được biết về câu hỏi này là những gì khung của một công ty nói về nó. Hồ sơ công bố phía bên kia mỏng hơn. Không có nghiên cứu về Rust trong các bài được lập chỉ mục của nhà cung cấp chủ trì công trình này. Lập trường đã bình duyệt gần nhất từ phía đó lập luận cho việc làm vệ sinh lỗi bộ nhớ trở nên thực tế. Nó không lập luận thay vệ sinh đó bằng một bảo đảm lúc biên dịch [37]. Đó là chiến lược mà luận văn này thử đối chứng.



Hình 2.2: Các nghiên cứu lân cận kiểm soát gì, và để gì biến thiên. Mỗi hàng là một thuộc tính mà một câu trả lời có kiểm soát sẽ cần, và mỗi cột là một nhóm nghiên cứu. Cột RAN nhà cung cấp là các luận văn Ericsson trừ Emberg và Wadsten, những người có cột riêng vì họ giữ thiết kế sát hơn.

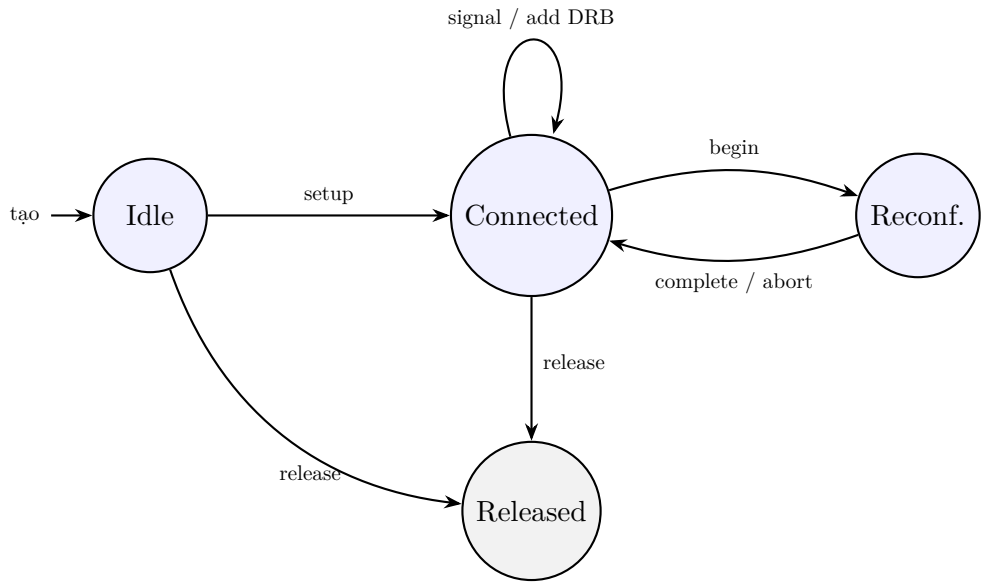
Điều còn thiếu trong công trình trước là một phép thử trực tiếp, có kiểm soát, trên một phần mặt điều khiển điển hình. Các câu hỏi mở như sau. Rust có giữ độ trễ đuôi khi tải quá lớn đối với C++ hiện đại khi mô hình luồng, thao tác và bố cục dữ liệu được giữ cố định? Nó loại những lớp lỗi nào, và phần còn lại bị bắt ở giai đoạn nào? Mô hình an toàn của nó tốn gì về thông lượng và về CPU cùng bộ nhớ? Áp dụng từng bước tốn gì trên thực tế? Đóng khoảng trống này là mục đích của bốn câu hỏi nghiên cứu. Chương tiếp theo neo chúng vào phần đang được nghiên cứu, các lớp lỗi, và thời gian đang bị đe dọa. Nó cũng nêu phương pháp.

3 Phương pháp nghiên cứu

Chương trước nêu khoảng trống. Chương này nêu cách nghiên cứu đóng nó.

3.1 Phân hệ Quản lý Ngữ cảnh UE

Phần được nghiên cứu là một dịch vụ Quản lý Ngữ cảnh UE. Nó được mô hình hoá theo một dịch vụ gNB-CU-CP thực của Nokia, sở hữu vòng đời ngữ cảnh UE, thực thi thủ tục RRC [4], và di động cùng chuyển giao. Đó là các chức năng mà thiết kế 3GPP giao cho phần mặt điều khiển của một gNB tách (Mục 2.1). Hình 3.1 cho thấy vòng đời ngữ cảnh UE mà cả hai bản mã hoá cùng một cách. Khối lượng hỗn hợp của Mục 5.2 đưa mỗi UE đi qua máy trạng thái đó.



Hình 3.1: Máy trạng thái vòng đời ngữ cảnh UE. Các thao tác không hợp lệ với trạng thái hiện tại trả về lỗi thay vì đổi trạng thái.

UECM là công việc tầng 3 RAN cốt lõi. Nó có những đặc tính làm mã mặt điều khiển đang chạy trở nên khó. Mỗi UE có trạng thái thay đổi. Vòng đời thao tác chặt. Nhiều UE chạy cùng lúc. Mục tiêu trễ là những mục tiêu hệ thống 5G đặt cho thủ tục mặt điều khiển [5, 27]. Phát hiện về cách một ngôn ngữ xử lý những đặc tính đó liên quan nhất tới các dịch vụ mặt điều khiển nhiều luồng, có trạng thái khác cùng loại. Phần này cũng đủ nhỏ để đặc tả trọn vẹn và xây hai lần. Đó là điều làm một phép so sánh có kiểm thử tương đương trở nên khả thi.

Giao diện nhỏ và được xây quanh vòng đời. Nó có thể tạo và giải phóng một ngữ cảnh. Nó có thể thiết lập một kết nối. Nó có thể xử lý một thông điệp báo hiệu trong phiên. Nó có thể thêm một data radio bearer. Nó có thể bắt đầu, hoàn tất, hoặc huỷ một tái cấu hình. Nó có thể tra trạng thái của một ngữ cảnh dưới tải. Mỗi thao tác được định nghĩa cho mỗi trạng thái của Hình 3.1. Một thao tác bất hợp pháp trả về lỗi có kiểu và để ngữ cảnh không đổi. Tập đóng đó cho phép cùng các kiểm thử và cùng khối lượng đưa cả hai bản đi theo cùng các đường.

RQ2 cần một danh mục, không phải một danh sách mối nguy lỏng. Một danh mục được xây từ tập thao tác này: 14 lớp trong hai nhóm bảy. Bảy lớp đến từ bên trong vùng mà Rust an toàn tuyên bố cai quản. Bảy lớp được chọn vì Rust an toàn được kỳ vọng không ngăn chúng. Mỗi lớp có kỳ vọng được ghi trước khi các chương trình chạy. Mục 5.3.4 báo cáo mỗi lớp đã làm gì.

Đuôi là chỉ số vì hai đặc tính của phần này. Một hành động người dùng nhìn thấy, thiết lập cuộc gọi hoặc chuyển giao, thành nhiều thao tác mặt điều khiển phụ thuộc nhau. Thành viên chậm nhất chặn hoàn tất. Một site bận đưa hàng nghìn thủ tục như vậy mỗi giây. Vì vậy một điểm đuôi xa được chạm đủ thường để có ý nghĩa. Dạng thất bại là lý do thứ hai. Một dịch vụ thiết kế này theo tải đưa vào với độ trễ phẳng cho đến khi một luồng xử lý bão hoà. Rồi độ trễ nhảy mạnh trong một bước tải. Đó là lý do nghiên cứu này quét qua bão hoà.

3.2 Hướng tiếp cận nghiên cứu

Nghiên cứu chạy thí nghiệm. Nó cũng xây thứ nó đo: một cặp bản khớp nhau, cộng khung đo đưa chúng chạy. Cả hai được xây từ một đặc tả cố định. Bằng chứng đến từ ba phương pháp tách biệt. Một quét tải trả lời RQ1 và RQ3. Tiêm lỗi có kiểm soát trả lời RQ2. Một nguyên mẫu di trú FFI trả lời RQ4. Một điểm yếu ở một trong số chúng, chẳng hạn máy chủ có thể dịch chuyển đuôi bao nhiêu, không lan sang phần còn lại của các tuyên bố.

Mỗi câu hỏi hỏi một kết quả phụ thuộc vào một biến, ngôn ngữ. Ngôn ngữ phải là khoảng cách có kế hoạch duy nhất giữa hai đối tượng thử. Điều đó loại trừ cách so sánh ngôn ngữ phổ biến nhất. Ở dạng đó hai chương trình viết độc lập được đo, và khoảng cách được lấy là ngôn ngữ. Khoảng cách đó cũng mang thiết kế, thư viện, và kỹ năng của tác giả. Xây cả hai phía từ một đặc tả loại nhiễu loạn đó. Ở nơi câu hỏi trả lời được bằng số, nó dùng số. Ở nơi không thể, câu trả lời vẫn theo một kế hoạch cố định, bằng lời. Đó là cách nghiên cứu này xử lý loại bảo đảm trong RQ2. RQ4 hỏi về chi phí thiết

kế của giao diện lập trình ứng dụng (API).

Công trình trước gần nhất là năm luận văn do nhà cung cấp lưu trữ của Mục 2.3. Chúng không xuất hiện trong cơ sở dữ liệu trích dẫn nào. Chúng phải được tìm bằng cách tra kho lưu trữ của từng trường bằng tay. Cùng các tìm kiếm đó không tìm thấy luận văn Phần Lan nào về Rust trong bối cảnh truy nhập vô tuyến. Chúng không tìm thấy bài được lập chỉ mục nào nhắc Rust từ nhà cung cấp chủ trì công trình này. Tìm kiếm không trả về nghiên cứu nào đo đuôi của một bản Rust và một bản C++ của cùng một phần mặt điều khiển nhiều luồng, có trạng thái, dưới tải quá lớn.

Bảng 3.1 ánh xạ mỗi câu hỏi nghiên cứu tới phương pháp, hiện vật, dữ liệu và chỉ số trả lời nó. RQ1 và RQ3 dùng chung H1, diễn đạt để phủ một hoà tốc độ theo cả nghĩa trễ và nghĩa tài nguyên. Cách diễn đạt đó hoá ra là điểm yếu chính của giả thuyết. RQ2 không phụ thuộc máy chủ. RQ4 phụ thuộc theo một trục. Thời gian biên dịch là một phép đo thời gian. Vì vậy nó được đọc như tỉ số giữa hai chuỗi công cụ đo luân phiên trên một máy.

Bảng 3.1: Cách mỗi câu hỏi nghiên cứu được trả lời. RQ1 và RQ3 mang hai phương pháp chứ không phải một: quét đo vòng khứ hồi mà một triển khai sẽ trải, và tách lõi khỏi bàn giao đo phần mà ngôn ngữ chịu trách nhiệm.

RQ	Phương pháp	Hiện vật / dữ liệu	Chỉ số chính
RQ1	Quét tải tới quá tải, lặp đan xen	Khung đo, histogram độ trễ	trung vị, p99/p99.9/p99.99, và cực đại so với tải đưa vào
RQ3	Cùng quét, thu tài nguyên	Mẫu `getrusage` mỗi lần chạy	kích thước tập trú (RSS) đỉnh, CPU, thông lượng
RQ1, RQ3	Gọi lõi trực tiếp, rồi qua dịch vụ	Cặp microbenchmark khớp	thao tác đối với bàn giao giữa luồng
RQ2	Tiêm lỗi có kiểm soát	Danh mục lỗi, ba bộ dò lỗi C++, `rustc`, Loom/Shuttle, Miri	biên dịch / lúc chạy / không phát hiện
RQ4	Nguyên mẫu di trú FFI	Máy chủ C++ + đường nối C-ABI Rust	kích thước đường nối, phạm vi `unsafe`, chi phí mỗi lần gọi

3.3 Phương pháp theo từng câu hỏi nghiên cứu, tóm tắt

RQ1 và RQ3 được trả lời bằng một quét tải vòng hở từ tải nhẹ đến vượt xa bão hoà. Vòng hở nghĩa là bộ sinh giữ lịch gửi của nó thay vì chờ trả lời [38]. Bốn biến thể build được đan xen trong mỗi lần lặp. Một phép đo thời gian thứ hai tách thao tác lõi khỏi bàn giao giữa luồng quanh nó. Vòng khứ hồi đầu-cuối gói hai chi phí thuộc hai loại. Chỉ một trong hai có thể khác giữa các ngôn ngữ. Một phép đo thứ ba lấy hàng đợi tranh chấp và tay cầm trả lời riêng. Vì vậy nguyên nhân quá tải có thể được thử chứ không chỉ được lập luận.

RQ2 được trả lời bằng tiêm lỗi có kiểm soát [39]. Mỗi lớp lỗi là một chương trình nhỏ riêng ở cả hai ngôn ngữ. Phía C++ được xây và chạy dưới AddressSanitizer. Nó cũng chạy dưới ThreadSanitizer và UndefinedBehaviorSanitizer. Phía Rust an toàn được biên dịch và, nơi nó biên dịch, chạy dưới một hạn thời gian bị chặn. Không bộ phân tích tĩnh nào được chạy, cố ý. Các cột C++ khi đó báo cáo công cụ mà nghiên cứu này đã chạy. Chúng không báo cáo trần của những gì công cụ C++ có thể tìm. RQ4 được trả lời bằng một nguyên mẫu chạy được theo hướng một cuộc di trú thực đi. Một máy chủ C++ gọi một lõi Rust không đổi qua ABI C. Chi phí mỗi lần gọi được đo trên một thao tác thực chứ không suy từ một lần vượt trống. Lỗi bộ nhớ vượt ngôn ngữ nằm ở đường nối đó [40].

3.4 Tính công bằng như một biến được kiểm soát

Một phép so sánh ngôn ngữ chỉ tốt bằng các biến nó giữ cố định. Đặc tả được cố định trước khi bản nào tồn tại. Mô hình luồng, bộ cấp phát và số arena của nó, mức tối ưu, hàm băm, phương pháp histogram và lịch tải được khớp giữa hai phía. Chúng không bao giờ được tinh chỉnh theo từng phía. Nơi hai ngôn ngữ không thể viết giống nhau, khoảng cách được ghi lại chứ không được hoà. Hai phía được chứng minh khớp bằng cách diff nhật ký khớp theo từng thao tác. Một hàng của bảng đó không có đối chứng. Một người viết cả hai bản và không thành thạo như nhau ở cả hai. Mục 6.5 coi đó là mối đe dọa nội tại chính của nghiên cứu.

3.5 Xử lý thống kê

Thiết kế thống kê được cố định trước. Họ các phép so sánh được thử được liệt kê trước chiến dịch. Vì vậy một mức tải chọn sau khi nhìn một quét không thể vào họ. Mỗi phép so sánh là một kiểm định hoán vị ghép cặp chính xác trên 20 lần lặp đan xen, với khoảng tin cậy bootstrap [41], hiệu chỉnh Holm trên cả họ [42]. Các sàn mà kiểm định có thể đạt (0.0000019 chưa hiệu chỉnh, 0.000027 đã hiệu chỉnh) được nêu để một kết quả null phân biệt được với một giới hạn toán. Hai đại lượng được giữ ngoài họ. Một là độ dao động từ lần gọi này sang lần gọi khác, mà không câu hỏi nghiên cứu nào hỏi. Đại lượng kia là đuôi xa p99.99 gộp. Đơn vị của nó là thao tác, không phải lần chạy. Nó khi đó không có khoảng giữa các lần lặp để thử.

3.6 Tái lập và hiện vật nghiên cứu

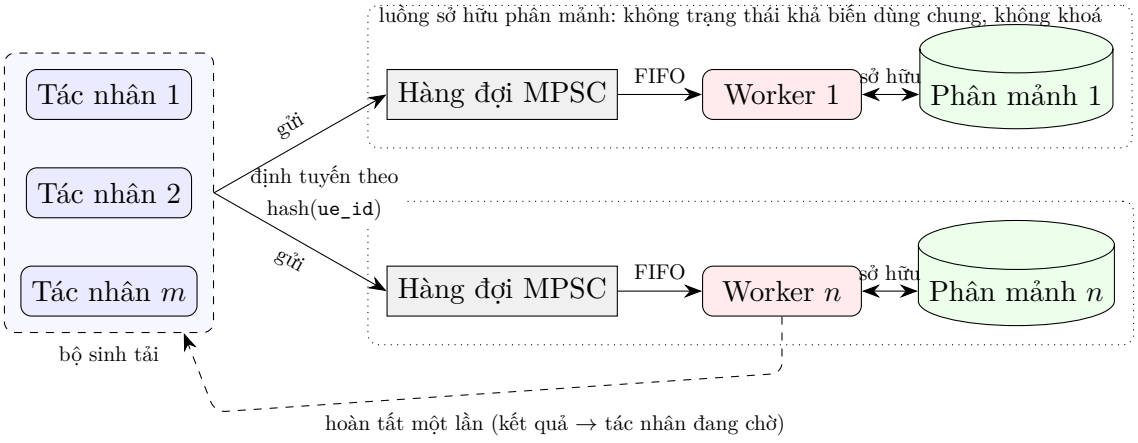
Một bên thứ ba có thể xây lại kết quả mà không cần mã riêng của Nokia. Các bản được đưa chạy hoàn toàn bởi đặc tả công khai đã cố định. Khung đo, danh mục lỗi và bộ chạy của nó, và nguyên mẫu FFI được công bố cùng luận văn. Mỗi hình dữ liệu đo được một kịch bản xây lại từ nhật ký thô. Mỗi số được trích trong kết quả cũng vậy. Kịch bản đó là `bench/refresh_thesis.sh`. Tuyên bố đó không phủ các sơ đồ vẽ tay, vốn giải thích một khái niệm chứ không báo cáo một kết quả. Và chạy cùng các kịch bản trên chiến dịch của một máy khác xây lại hiện vật theo cùng thủ tục, không với cùng giá trị.

Các nhật ký mà kịch bản đọc là một phần của gói. Mỗi chiến dịch có kịch bản riêng và tiền tố macro riêng, nên một hình từ chiến dịch này không thể chạm một câu về chiến dịch kia. Thiết lập máy chủ của mỗi lần chạy được ghi và lưu cạnh dữ liệu chiến dịch. Không gì trong các chương kết quả được gõ từ những tệp đó bằng tay.

4 Thiết kế và hiện thực

Không gì được xây cho đến khi đặc tả được cố định. Nó chốt giao diện lập trình công khai, máy trạng thái UE, bố cục dữ liệu và các bước trên đó, khoá và kích thước phân mảnh, và mô hình luồng. Cả hai bản đều bám theo nó. Phép thử khi đó đo ngôn ngữ, không phải một khoảng cách thiết kế hay kỹ năng. Bản thân đặc tả là một đầu ra của nghiên cứu. Nó là kiểm soát chính chống thiên lệch của người viết.

Một mô hình luồng được dùng ở cả hai phía: không chia sẻ trạng thái và một luồng mỗi phân mảnh. Bố cục đó được biết là tự nó đổi độ trễ đuôi [43]. Không gian định danh được tách thành các phân mảnh. Mỗi phân mảnh thuộc một luồng xử lý. Chỉ luồng đó đọc hoặc đổi nó. Vì vậy không khoá nào canh trạng thái ngữ cảnh. Bên gọi gửi một yêu cầu tới hàng đợi bị chặn của phân mảnh sở hữu. Chúng chờ trên một tay cầm trả lời một lần. Hàng đợi chặn khi đầy. Điều đó là cố ý. Một quy tắc bỏ sẽ che quá tải. Nghiên cứu này cần quá tải hiện ra như độ trễ. Trạng thái khả biến dùng chung duy nhất mà bản nào cho phép là một bộ đếm nguyên tử. Không luồng điều khiển nào phụ thuộc giá trị của nó. Hình 4.1 cho thấy luồng dữ liệu. Có một hàng đợi nhiều tác nhân sinh, một tác nhân tiêu thụ (MPSC) mỗi phân mảnh. Luồng xử lý của nó rút theo vào trước ra trước (FIFO).



Hình 4.1: Kiến trúc không chia sẻ, một luồng mỗi phân mảnh. Các tác nhân sinh gửi vào một hàng đợi MPSC bị chặn mỗi phân mảnh, và luồng xử lý của phân mảnh đó rút hàng.

Tính công bằng được giữ dưới thiết kế cũng như trong thiết kế. Cả hai bản build liên kết cùng `jemalloc`. Cả hai giữ một bản build bộ cấp phát hệ thống làm đối chứng. Cả hai biên dịch ở cùng mức tối ưu. Cả hai ghi độ trễ qua một bộ histogram dải động cao (HDR) được lập cùng một cách [44]. Băm được chốt ở cả hai phía. SplitMix64 gán phân

mảnh. FNV-1a, hàm băm Fowler–Noll–Vo, được dùng bên trong bảng. Hàm băm mặc định của C++ trên một số nguyên thường chỉ là chính số đó. Của Rust mạnh là cố ý. Để nguyên như chúng đến, hai thư viện chuẩn sẽ là phép đo.

Nơi các ngôn ngữ không thể đáp một nhu cầu cùng một cách, khoảng cách được viết vào nhật ký thành ngữ bắt buộc rồi được đo. Hai trong số các khoảng cách đó mang kết quả quá tải của Mục 5.3.2. Một là hàng đợi bị chặn. Rust lấy nó từ `sync_channel`. C++23 vẫn chưa có nguyên thủy đó, nên C++ xây một cái từ một mutex và hai biến điều kiện. Cái kia là tay cầm trả lời, `std::promise` đối với một kênh `mpsc` mới mỗi yêu cầu. Một khoảng cách thứ ba mang kết quả thao tác lõi của Mục 5.3.1: container mỗi phân mảnh là bảng băm chuẩn của chính ngôn ngữ đó. Một khoảng cách thứ tư là mã hoá yêu cầu. C++ không có kiểu liệt kê mang payload theo thành ngữ này, nên một yêu cầu thành một struct có nhãn giữ mỗi trường mà thao tác nào cũng có thể cần, 160 byte đối với 96 của Rust. Kết quả ròng là một lõi Rust dài hơn 18% về dòng mã. Độ phức tạp cyclomatic của nó thấp hơn 15% [45]. Thư viện Rust không chứa `unsafe` nào cả.

Tương đương được thử chứ không được giả định. Các vector trung lập ngôn ngữ chạy ở cả hai bản. Nhật ký của chúng được diff theo từng thao tác. Một diff chéo không phân biệt “cả hai đúng” với “cả hai sai cùng một cách”. Một bản thứ ba bằng Python đóng vai tiên tri độc lập. Nó phải tái tạo tất cả 223 kỳ vọng suy bằng tay trước khi được sinh kỳ vọng của riêng mình. Một chiến dịch 793 000 thao tác được sinh không tìm thấy lệch. Phía C++ chạy dưới AddressSanitizer, ThreadSanitizer và UndefinedBehaviorSanitizer. Phía Rust chạy dưới Miri. Một kết quả bộ dò lỗi sạch bị chặn bởi mã thực sự đã chạy.

5 Thiết lập thí nghiệm và đánh giá

Chương này bắt đầu với các máy thử và khối lượng công việc. Kết quả rồi theo chế độ tải, rồi theo các câu hỏi nghiên cứu còn lại.

5.1 Thiết lập hệ thống thử

Một số số trong chương này đến từ trình biên dịch và mã. Các số khác phụ thuộc máy chúng chạy trên. Mọi số tốc độ ở đây đến từ một máy chủ vật lý chuyên dụng, một socket AMD EPYC 7443. Mỗi nhân chạy một luồng. Đa luồng đồng thời là off. Máy chạy Ubuntu 24.04.4 LTS trên nhân 6.17.0-23-generic. Linux có thể tăng và giảm xung mỗi nhân khi tải và nhiệt đổi. Xung được giữ cố định: bộ điều tần là performance, và boost tắt. Benchmark chỉ được dùng các nhân 2–17. Không có hypervisor, nên chương trình chạy trên chính máy chứ không trong một máy ảo. Trên các nhân đó nhân hệ điều hành tắt ngắt thời gian định kỳ (`nohz_full`). Những ngắt đó nếu không sẽ hiện trong các số độ trễ. Nghiên cứu không dùng `isolcpus`. Cờ đó cũng ngăn nhân rải việc trên các nhân được phép. Mọi luồng benchmark khi đó đáp xuống một nhân.

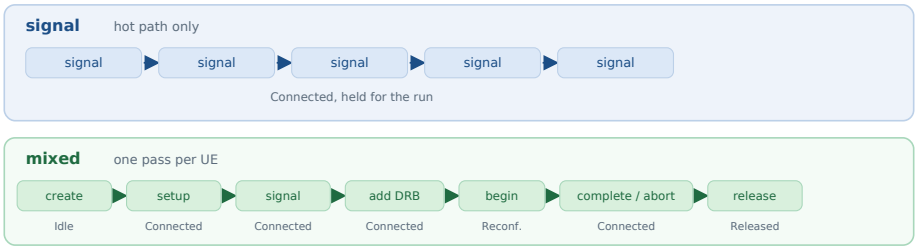
Các trình biên dịch không phải những cái cài trên máy chủ. Cả hai bản được xây và chạy trong một ảnh container mang Rocky Linux 8.10 (Green Obsidian) với `g++ 15.2.1` và `rustc 1.97.1`. Ảnh chốt phiên bản chính `gcc-toolset` chứ không phải bản phát hành trong đó. Chiến dịch trước trên một máy ảo vì vậy biên dịch với `g++ 15.1.1`, sớm hơn một bản phát hành GNU Compiler Collection (GCC) 15. Một container vẫn dùng nhân của máy nó ngồi trên. Hai chiến dịch khi đó khác nhau về phần cứng, về nhân (6.17.0-23-generic đối với 7.0.2-1.el8.elrepo.x86_64), và về bản phát hành trình biên dịch đó.

Cho đến muộn trong công việc, chiến dịch chạy trên một máy ảo thay vì vậy: Kernel-based Virtual Machine (KVM) trên một Intel(R) Xeon(R) CPU E5-2680 v4, không có các kiểm soát nhân ở trên. Trên máy đó khoảng tin cậy 95% trên p99 của một biến thể đơn trải 1480 μs, trong khi giá trị p99 của bốn biến thể lệch 293 μs. Nhiễu trên một biến thể lớn hơn khoảng cách giữa cả bốn. Chiến dịch đó được giữ. Nó dùng cùng thiết kế với chiến dịch này, nên hai bên có thể được so để hỏi phát hiện nào thuộc về mã. Một quét thứ ba trên một máy vật lý thứ hai AMD EPYC 7443P dùng số phân mảnh, lưới tải, số lần lặp và trình biên dịch khác. Nó thử liệu các phát hiện có sống sót những thay

đổi đó cùng nhau, không phải từng cái tách riêng.

5.2 Khối lượng công việc và phép đo

Khối lượng được làm cho thí nghiệm. Các bước nó đi lấy từ quy tắc 3GPP [1, 4]. Một bộ lái luôn gửi cùng chuỗi đưa mỗi UE qua vòng đời kết nối mà các quy tắc đó định. Khối lượng *signal* trước hết kết nối một tập UE, rồi gửi cùng thao tác báo hiệu hết lần này đến lần khác. Điều đó cô lập đường phổ biến. Khối lượng *mixed* đưa mỗi UE qua tạo ngữ cảnh, thiết lập, báo hiệu, thiết lập bearer, tái cấu hình hoặc chuyển giao, và giải phóng. Một thứ tự ngẫu nhiên cố định làm cả hai ngôn ngữ thấy cùng số đếm mỗi loại thao tác cho một hạt giống cho trước. Đó cũng là một kiểm tra rằng hai bản đã làm cùng việc. Hình 5.1 cho thấy hai khối lượng và chuỗi mà khối mixed đi.



Hình 5.1: Hai khối lượng, và vòng đời mà khối mixed đi. Signal lặp đường phổ biến, và mixed đưa mỗi UE qua chuỗi thủ tục vẽ dưới.

Chuỗi thao tác khớp quy tắc 3GPP. Tần suất mỗi loại xuất hiện thì không. Một cell thực chuyển giao thường đến mức nào phụ thuộc site và giờ trong ngày, và không con số công khai nào chốt nó. Một hỗn hợp không khớp một cell thực dịch cả hai ngôn ngữ cùng một lượng. Nó giới hạn mức các số đi xa được. Nó không làm phép so sánh bất công. Bộ gửi giữ lịch của nó dù dịch vụ trễ (vòng hở), và histogram được hiệu chỉnh, nên một tắc không che các trả lời thiếu (thiên lệch bỏ sót phối hợp) [38]. Mỗi mức tải được lặp 20 lần. Bốn biến thể ngôn ngữ và bộ cấp phát được đan xen trong mỗi lần lặp. Mỗi điểm báo cáo là một trung vị với khoảng tin cậy bootstrap 95% [41, 46].

Tải dùng cho các hình tài nguyên là 56 kilo-thao-tác mỗi giây (kops/s). Đó là một giá trị được chọn, không phải một giá trị tính từ quét. Quy tắc đầu tiên nghiên cứu này dùng là mức tải bận nhất mà p99 ở dưới 5 ms. Trên máy ảo quy tắc đó đáp xuống nhiễu của máy chủ. Trên một máy yên nó leo cho đến khi dịch vụ không còn theo kịp. Nó được giữ chỉ như một kiểm tra. Cao nguyên thông lượng thấp nhất trong bốn biến thể là 392 kops/s, đạt khi tải đưa vào là 392 kops/s.

5.3 Kết quả

Các kết quả an toàn và phần lớn kết quả di trú không phụ thuộc máy nào được dùng. Các kết quả tốc độ và bộ nhớ thì có. Mỗi hình dưới đây được đo trên máy chủ của Mục 5.1 trừ khi chú thích nói khác.

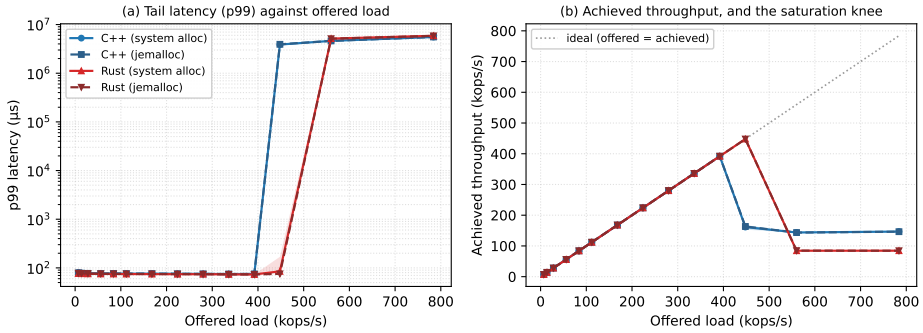
5.3.1 Dưới bão hoà (RQ1, RQ3)

Khi dịch vụ vẫn theo kịp tải đưa vào, trung vị của bốn biến thể nằm trong 1.6 μs của nhau, 2.2%. Một kiểm định ghép cặp vẫn tìm thấy khác biệt 1.3 μs nghiêng về Rust dưới cả hai bộ cấp phát. Hình 5.2 cho thấy quét đứng sau điều đó. Các phân vị cao hơn nay cũng đọc được. Khoảng rộng nhất trên p99 của một biến thể đơn trải 0.8 μs. Giá trị p99 của bốn biến thể lệch 1.6 μs. Vì vậy nhiễu trên một cái nhỏ hơn khoảng cách giữa chúng. Trên máy ảo cùng hai đại lượng đó là 1480 μs đối với 293 μs, theo chiều ngược. Cả hai hàng p99 và cả hai hàng p99.9 vượt hiệu chỉnh Holm, 4 hàng tất cả, cùng chiều với trung vị. Bảng 5.1 báo cáo p99.9 và cực đại tại tải đại diện. Cực đại được giữ ngoài họ được thử, như p99.99 gộp.

Bảng 5.1: Độ trễ mỗi thao tác, tính bằng micro giây, tại 56 kops/s. Trung vị trên 20 lần lặp nằm trên khoảng bootstrap 95% của nó. †Cột p99.99 được gộp giữa các lần lặp, nên không có khoảng. Cực đại được giữ ngoài họ được thử vì cùng lý do.

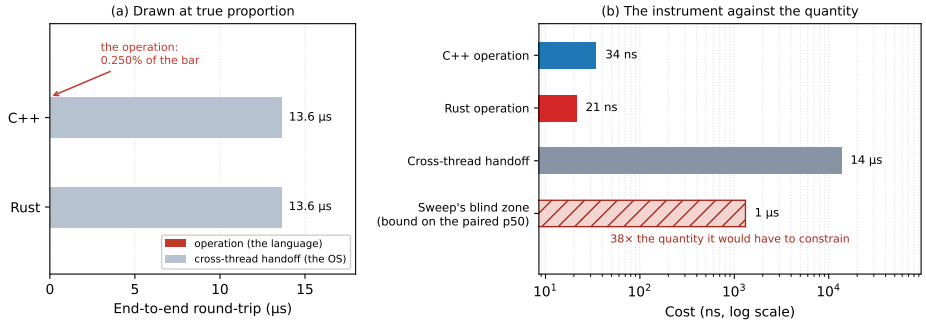
			(µs)		
Variant	p50	p99	p99.9	p99.99†	max
C++ (system alloc)	73.7	76.4	81.2	142.0	243.1
	[73.5, 73.7]	[76.2, 77.0]	[80.8, 81.6]		[239.2, 275.8]
C++ (jemalloc)	73.5	76.6	80.2	176.4	246.3
	[73.3, 73.6]	[76.4, 76.9]	[79.5, 82.0]		[228.8, 259.5]
Rust (system alloc)	72.4	75.0	77.3	92.1	145.2
	[72.3, 72.5]	[74.7, 75.2]	[77.0, 78.7]		[115.9, 178.2]
Rust (jemalloc)	72.1	75.6	77.8	89.5	121.4
	[72.1, 72.3]	[75.6, 75.7]	[77.7, 77.9]		[101.8, 179.6]

Đuôi xa cũng đọc được ở đây. Histogram của các lần lặp được gộp, cho 3 360 051 đến 3 360 084 thao tác mỗi biến thể. Hai biến thể Rust nằm dưới cả hai biến thể C++. Cặp rộng nhất lệch 87 μs. Biến thể Rust chậm nhất vẫn nằm 50 μs dưới biến thể C++ nhanh nhất. Một máy vật lý thứ hai ở số phân mảnh khác đồng ý, chạy 147–228 μs cho các biến thể C++ đối với 88–115 μs cho các biến thể Rust, không chồng. Một giới hạn thứ hai không đến từ máy. Không máy chủ nào xoá được giới hạn đó. Một vòng khứ hồi



Hình 5.2: Quét tải, cả hai tấm đối với tải đưa vào. Đường là trung vị trên 20 lần lặp, và dải là khoảng tin cậy bootstrap 95%.

đầu-cuối chỉ dành 0.250% thời gian cho công việc ngôn ngữ thực hiện. Phần còn lại là đưa yêu cầu sang luồng khác, và thời gian đó được tính cho hệ điều hành. Khi công việc bên trong ngôn ngữ nhỏ như vậy, thời gian vượt biên luồng chiếm ưu thế [47]. Quét này không thể giới hạn một hiệu ứng ngôn ngữ theo chiều nào, điều mà Hình 5.3 đặt cạnh những gì quét vẫn còn cho thấy. Đo riêng, thao tác lõi nhanh hơn 1.6× ở Rust. Nguyên nhân khả dĩ là cách bảng băm chuẩn được bố trí, không phải mô hình an toàn.



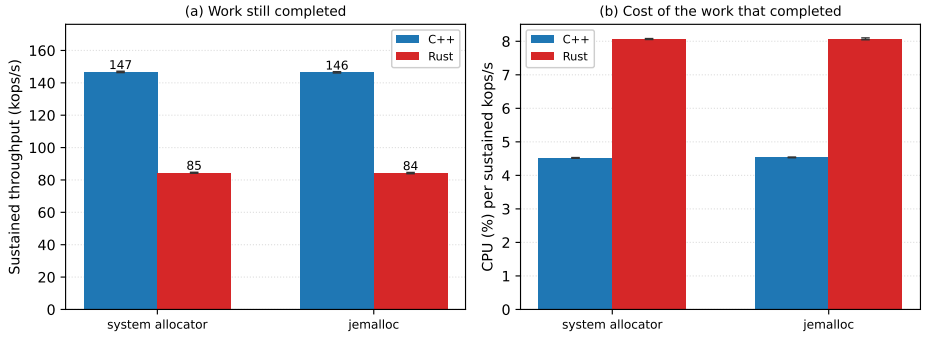
Hình 5.3: Chi phí của một vòng khứ hồi đầu-cuối đi đâu. **(a)** Vòng khứ hồi ở tỉ lệ thật. **(b)** Cả hai phép đo thao tác nằm trong vùng mù của quét theo hệ số 38.

5.3.2 Qua bão hoà (RQ1)

Khi tốc độ đưa vào cao hơn dịch vụ chịu được, bốn biến thể ngừng theo kịp ở các tải khác nhau. Các bản Rust giữ đến 448 kops/s. Các bản C++ giữ đến 392. Vì vậy tại 448 kops/s Rust vẫn phục vụ 448 kops/s nơi C++ đã rơi xuống 162. Mức tải đó được mô tả chứ không được thử, vì nó được tìm sau khi dữ liệu đã có.

Sâu hơn, tại 784 kops/s đưa vào, thứ tự đảo. C++ vẫn hoàn thành 147 kops/s nơi Rust hoàn thành 84, một hệ số 1.7, trong 20/20 lần lặp dưới cả hai bộ cấp phát. Giá trị p đã hiệu chỉnh Holm nhỏ đến mức họ kiểm định này có thể đi. Biến thể chậm hơn không

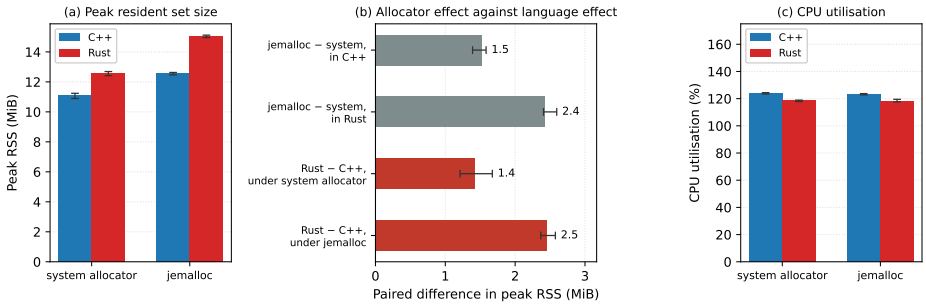
chỉ làm ít việc hơn. Nó dùng 1.8 lần CPU cho mỗi thao tác nó hoàn thành, dấu hiệu của nỗ lực bỏ ra mà không hoàn tất gì. Khi hàng đợi bị chặn và tay cầm hoàn tất được đo riêng, thứ tự đó không có ý nghĩa, nên nguyên nhân nằm trên đường rộng hơn đưa việc giữa các luồng. Đây là hiệu ứng ngôn ngữ lớn nhất trong họ được thử. Một phép so sánh dừng ở tải mà dịch vụ vẫn đáp sẽ chỉ theo chiều kia. Hình 5.4 đặt những gì mỗi biến thể vẫn hoàn thành cạnh những gì nó tốn để hoàn thành.



Hình 5.4: Hành vi tại 784 kops/s đưa vào. Cột là trung vị trên 20 lần lặp với khoảng bootstrap 95%, nhóm sao cho hai cột trong một nhóm chỉ khác ngôn ngữ. CPU trong (b) được cộng trên các luồng.

5.3.3 Dùng tài nguyên (RQ3)

Bộ nhớ và CPU tách các biến thể nơi độ trễ không tách được. Hai hiệu ứng cỡ tương đương nằm trong RSS đỉnh. Bản build Rust dùng 1.4–2.5 MiB bộ nhớ đỉnh nhiều hơn bản build C++ dưới bộ cấp phát nào. Liên kết `jemalloc` nói chung tốn 1.5–2.4 MiB ở ngôn ngữ nào. Cả hai đều thực, và Hình 5.5 đặt cặp đó cạnh RSS đỉnh mà chúng cộng thành. Hiệu ứng ngôn ngữ giữ chiều và cỡ trên cả ba máy được đo. Của bộ cấp phát là 13 lần hiệu ứng ngôn ngữ trên máy ảo nghiên cứu này dùng trước (32.5–34.3 MiB), nên tỉ số của chúng phụ thuộc máy. Khác biệt ngôn ngữ là chi phí cố định, không phải chi phí mỗi UE. Máy yên là nơi điều đó nhìn thấy được. Bộ đếm của chính `jemalloc` ở đó đặt +2.07 MiB nhiều hơn vào sổ sách arena và −0.16 MiB *ít hơn* vào cấp phát còn sống. Byte trú của nó thấp hơn −1.92 MiB. Đỉnh tiến trình cao hơn +1.74 MiB. Bộ nhớ thêm khi đó nằm ngoài bộ cấp phát, trong thời gian chạy mà một tiến trình Rust mang trước khi nó cấp phát. CPU đổi dấu theo tải. Tại tải đại diện 56 kops/s Rust dùng ít hơn tối đa 5.1 điểm phần trăm. Khi dịch vụ không còn theo kịp, CPU mỗi thao tác hoàn thành cao hơn nhiều.

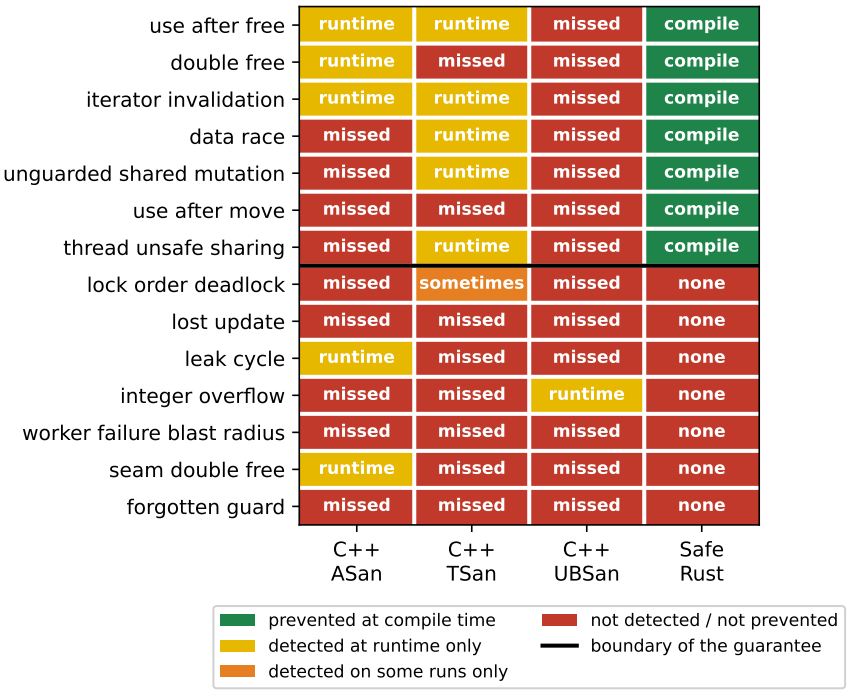


Hình 5.5: Dùng tài nguyên tại 56 kops/s, nhóm theo bộ cấp phát. Cột là trung vị trên 20 lần lặp với khoảng bootstrap 95%, và CPU trong (c) được cộng trên các luồng.

5.3.4 An toàn qua tiêm lỗi (RQ2)

An toàn được đo bằng cách tiêm 14 lỗi. Bảy lớp đến từ bên trong vùng Rust tuyên bố cai quản. Bảy lớp được chọn vì Rust an toàn được kỳ vọng *không* ngăn chúng. Mỗi lớp trong bảy lớp đầu là lỗi biên dịch ở Rust. Ở C++ mỗi lớp là mã hợp lệ. Lỗi chỉ hiện nếu một kiểm thử đưa nó chạy, và không bộ dò lỗi đơn nào phủ chúng. Use-after-move là trường hợp không bộ dò lỗi nào báo gì. Một đối tượng đã move vẫn là một đối tượng hợp lệ. Giá trị của nó không xác định. Vì vậy không có gì bất hợp pháp để phát hiện, và định danh đơn giản là mất. Cả bảy trường hợp biên đều biên dịch trong Rust an toàn và hành xử sai. Trên hai lớp trong số đó công cụ C++ là bên báo. LeakSanitizer gắn cờ chu trình tham chiếu mà Rust rò trong im lặng, vì rò là an toàn trong Rust theo thiết kế tường minh [48, 49]. UndefinedBehaviorSanitizer gắn cờ overflow mà Rust định nghĩa là quấn. ThreadSanitizer bắt đảo thứ tự khoá trong 5 trên 12 lần chạy, một tỉ lệ bản thân không ổn định. Nó chỉ ghi thứ tự khoá từ các khoá thành công. Nếu lần đảo đầu chờ mãi, chu trình không bao giờ đóng, nên công cụ không bao giờ báo.

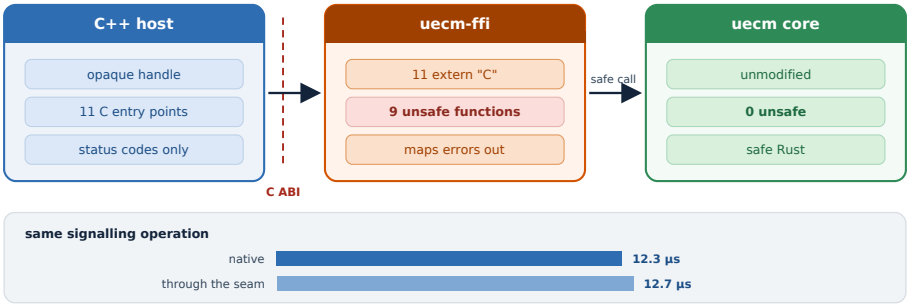
Double-free đường nối là double-free trong bảo đảm viết lại qua `unsafe`, không phải một lớp mới. Nửa C++ của nó không đổi, nên chỉ cột Rust dịch. Hai bộ kiểm tra mô hình Rust có báo cả hai lỗi biên gốc. Loom duyệt xen kẽ luồng một cách vững. Shuttle lấy mẫu những xen kẽ Loom không với tới [50]. Mỗi cái phải được viết bằng nguyên thủy của chính bộ kiểm tra trước, và thu nhỏ về cỡ mô hình. Miri báo double-free đường nối, nhưng nó diễn giải Rust và không chạy được máy chủ C++ mà lỗi nằm sau. LeakSanitizer và UndefinedBehaviorSanitizer báo mà không cần viết lại. Hai bộ kiểm tra mô hình chỉ báo các trường hợp biên tương tranh sau khi lỗi được viết lại vào nguyên thủy của chúng. Hình 5.6 vẽ cả danh mục như một ma trận kết cục. Resource Acquisition Is Initialization (RAII) gỡ việc quên mở khoá thông thường. Nó không gỡ lớp: cả hai chương trình quên guard đều treo, và công cụ của ngôn ngữ nào cũng không nói vậy.



Hình 5.6: Kết cục tiêm lỗi như một ma trận, sinh từ nhật ký phân loại.

5.3.5 Di trú (RQ4)

Đường nối di trú nhỏ trên mỗi trục được đo: 11 điểm vào C trong 231 dòng Rust. Mọi việc con trỏ thô bị nhốt trong 9 hàm. Lõi được để không `unsafe`. Không chi phí thêm nào được phát hiện đối với cùng cuộc gọi báo hiệu thực hiện trực tiếp. Điều đó giới hạn mọi thuế mỗi lần gọi ở 10.91% của cuộc gọi đó, và Hình 5.7 cho thấy cấu trúc mà giới hạn đó thuộc về. Ba chi phí áp dụng ngoài đường nối cũng được đo, và hai trong số đó mâu thuẫn với phản đối thường gặp khi áp dụng Rust. 54% của lần build sạch Rust là mã bên thứ ba. Mã nguồn của chính dự án biên dịch trong 0.8 lần CPU mà `g++` cần. Khoảng cách kích thước nhị phân 6.8× là một chính sách liên kết đảo chiều khi chính sách được giữ cố định. Thành phần được so sánh không có crate bên thứ ba và không có vị trí `unsafe` (0 crate, 0 vị trí `unsafe`).



Hình 5.7: Đường nối di trú FFI từ C++ sang Rust. Cùng thao tác báo hiệu tốn 12.3 µs trực tiếp và 12.7 µs qua đường nối, một khác biệt ghép cặp được giới hạn ở 10.91% của cuộc gọi đó. Một lần vượt không làm gì tốn 2.54 ns và chỉ giới hạn chính nó.

6 Thảo luận

6.1 Tóm tắt phát hiện

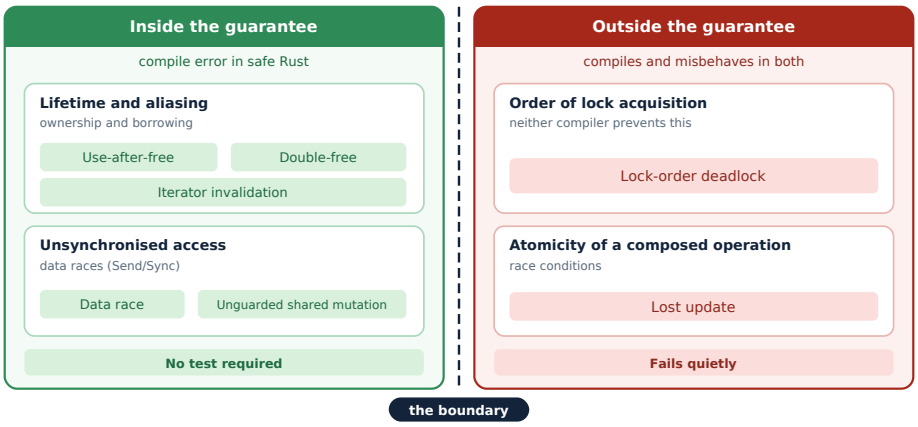
Câu trả lời cho RQ1 chia tại bão hoà. Dưới đó các biến thể khớp nhau đến 1.6 μs ở trung vị, và đuôi xa phân giải nghiêng về Rust. Vòng khứ hồi đầu-cuối chỉ là 0.250% công việc ngôn ngữ, nên nó không thể giới hạn một hiệu ứng ngôn ngữ. Cùng thao tác đo riêng nhanh hơn 1.6× ở Rust. Qua bão hoà chúng phân kỳ theo cả hai chiều. Các nhánh Rust giữ tốc độ đưa vào đến 448 kops/s đối với 392. Tại 784 kops/s C++ duy trì 1.7× thông lượng của Rust. RQ2 là kết quả đã chốt và không phụ thuộc máy chủ. Cả bảy lớp trong bảo đảm đều là lỗi biên dịch trong Rust an toàn. Cả bảy lớp biên đều biên dịch và hành xử sai ở cả hai ngôn ngữ. RQ3 là một phạt bộ nhớ 1.4–2.5 MiB tái hiện dưới cả hai bộ cấp phát và trên cả ba máy, với CPU đảo dấu qua hai chế độ tải. RQ4 là một đường nối nhỏ mà thuế mỗi lần gọi được giới hạn ở 10.91% của cuộc gọi đó. Chi phí thực là thiết kế API và đường cong học tập.

6.2 Thảo luận các câu hỏi nghiên cứu

Việc một triển khai nên làm gì với RQ1 phụ thuộc nơi nó kỳ vọng ngồi, không phụ thuộc một xếp hạng đơn. Một dịch vụ được định cỡ để chạy dưới điểm gối của mỗi nhánh nên chọn nhánh giữ tốc độ đưa vào xa hơn. Một dịch vụ mà một cơn bão báo hiệu sẽ đẩy qua mỗi điểm gối nên chọn nhánh vẫn hoàn thành nhiều việc hơn sau đó. Hai ưa thích đó hướng tới hai ngôn ngữ đối lập. Chỉ cái thứ hai được thử. Cái thứ nhất là mô tả, vì lý do Mục 5.3.2 đưa. Một phép so sánh dừng ở tải mà dịch vụ vẫn đáp sẽ đọc chiều ngược với cùng bộ máy.

Công trình trước gần nhất đo đại lượng này cho các thủ tục mặt điều khiển 5G. Nó tìm cùng hình dạng và truy nó về `async`/`await` không ngăn xếp [32]. Quy kết đó được chấp nhận ở đây. Hình dạng xuất hiện mà không có thời gian chạy bất đồng bộ ở phía nào và mô hình tương tranh được giữ cố định. Một thời gian chạy không thể là nơi duy nhất một hiệu ứng như vậy phát sinh. Dự đoán đăng ký trước cũng nêu nơi nhìn trước, kiểm biên và kiểm overflow [51, 52]. Phần đó của nó sai. Khoảng cách trung vị khớp với bằng chứng rằng các kiểm an toàn của Rust phần lớn được CPU hiện đại ra-khỏi-thứ-tự hấp thụ [12, 35]. Nó cũng yếu hơn vẻ ngoài. Thao tác lõi là 0.250% những gì quét đo.

Về an toàn, một số lỗi không thể chạm một nhị phân chút nào. Bảy lớp Rust không ngăn là một phần của câu trả lời, không phải một cảnh báo thêm sau. Một trình biên dịch từ chối xây một chương trình loại lỗi khỏi mỗi lần thực thi. Một bộ dò lỗi chỉ loại nó khỏi những lần chạy mà một kiểm thử bộ dò lỗi tình cờ đi. Cái tuyệt đối ở đó là phủ thực thi, không phải bề rộng của danh mục. Hình 6.1 nêu cơ chế mỗi bên của đường đó. Rust lý luận về aliasing chứ không về thứ tự chiếm. Một guard `Mutex` mà bộ kiểm mượn theo dõi vòng đời hoàn hảo không nói gì về việc hai luồng lấy khoá theo thứ tự tương thích. Phát hiện khoá chết tĩnh cho Rust là một bài toán nghiên cứu đang mở [53]. Lost update thoát ThreadSanitizer vì cùng lý do nó thoát bộ kiểm mượn. Nơi mỗi truy cập được đồng bộ đúng, không có truy cập không đồng bộ để tìm. Trong một nghiên cứu hiện trường 105 lỗi tương tranh thực, khoảng một phần ba các trường hợp không phải khoá chết vi phạm thứ tự lập trình viên định chứ không phải việc khoá [54]. Áp dụng Rust loại một hạng lỗi tương tranh và để nguyên hạng thất bại trong im lặng. Một kế hoạch di trú đọc "an toàn bộ nhớ" như "an toàn tương tranh" sẽ tính sai ngân sách rà soát. Lỗi an toàn bộ nhớ trong Rust dính `unsafe` trong mọi mục Common Vulnerabilities and Exposures (CVE) đã công bố trừ một lỗi trình biên dịch [55], đó là lý do giữ lõi không `unsafe` quan trọng, và lý do mã `unsafe` lan qua phụ thuộc là mối quan tâm rà soát [56].



Hình 6.1: Nơi bảo đảm lúc biên dịch của Rust dừng, các lớp nhóm theo cơ chế từ chối chúng.

Một trường hợp vượt quá các chương trình được tiêm. Một lỗ đặc tả cho phép một thao tác được gửi ngoài cửa sổ chạy của dịch vụ. Nó sống sót mỗi kiểm thử, mỗi vector tương đương và cả ba bộ dò lỗi vì không gì từng gọi đường đó. Cái biến hành vi không định nghĩa thành một thất bại chẩn đoán được ở phía Rust là một kiểm biên thông thường, không phải một chứng minh. Thuộc tính đáng ngân sách là "được kiểm mặc định", không phải "bị loại lúc biên dịch". Nguyên nhân gốc thuộc về phương pháp. Một cấm nêu ra mà không có cơ chế phía sau để mỗi bản thất bại theo thành ngữ của riêng

nó.

RQ3 tách theo tài nguyên và các tách không chỉ cùng chiều. Phạt bộ nhớ tái hiện dưới cả hai bộ cấp phát và trên một máy thứ hai với trình biên dịch khác. Tập trú đỉnh là cực đại cấp lần chạy, nên độ dao động phá một p99 không chạm nó. Đo so với hiệu ứng của chính bộ cấp phát, hai cái cùng bậc trên cả hai máy vật lý. Trên máy ảo, của bộ cấp phát lớn hơn một bậc. Hiệu ứng ngôn ngữ đi được. Tỉ số thì không. Máy yên cũng định vị chi phí ngoài bộ cấp phát, bằng sổ sách của chính bộ cấp phát. Một nghiên cứu gần báo dấu ngược, Rust thấp nhất và khoảng cách rộng khi tương tranh tăng [29]. Hai phát hiện đó khớp nhau khi các nhánh được đọc như hai thời gian chạy tương tranh. C++ của họ chạy trên Boost.Fibers, cấp một ngăn mỗi fiber. Rust của họ chạy trên Tokio, nơi một task là một future nhỏ cấp trên heap. Họ đo chi phí mỗi ngữ cảnh của hai thời gian chạy tương tranh. Hai nghiên cứu nữa khớp chiều tìm thấy ở đây [30, 57]. Về CPU dấu phụ thuộc tải. Một số đơn cho “chi phí CPU của mô hình an toàn Rust” sẽ trung bình hai chế độ đi ngược chiều.

RQ4 ra rẻ ở máy và đắt ở người. Một máy chủ C++ có thể gọi một lõi Rust không đổi qua một đường nối C-ABI mỏng mà chi phí trên một cuộc gọi báo hiệu thực được giới hạn ở 10.91% của cuộc gọi đó. Chi phí chiếm ưu thế là con người [58, 59]. Nhốt `unsafe` vào đường nối giới hạn những gì phải rà soát, không phải những gì có thể bị làm hỏng. Máy chủ và thành phần chia một không gian địa chỉ [60, 61]. Các trục build và dấu chân chỉ cùng chiều với đường nối. Một nhà cung cấp lo một trong hai nên kiểm ngân sách phụ thuộc trước khi đổ lỗi trình biên dịch. Trục duy nhất C++ dẫn thẳng là vòng sửa-biên dịch cho các thay đổi nằm trong một đơn vị dịch. Con số kích thước nhị phân 6.8× so hai chính sách liên kết, và thứ tự đảo khi chính sách được giữ cố định.

6.3 Xem lại các giả thuyết

Mục 1.4 cam kết ba giả thuyết trước mọi phép đo. Nêu cách xử chúng là điều tách một phép thử khỏi một mô tả.

H1 Mệnh đề trung vị giữ theo nghĩa thực tiễn. Kiểm định ghép cặp phân giải 1.3 μs, quá nhỏ để giới hạn một hiệu ứng ngôn ngữ. Mệnh đề đuôi xa không giữ, và quá tải, điều mà cách diễn đạt không bao giờ nêu tên, là nơi các ngôn ngữ thực sự tách. Chiến dịch được báo cáo nay phân giải đuôi đó, nghiêng về Rust, và một máy vật lý thứ hai đồng ý về chiều. H1 đúng về nơi một khác biệt dưới bão hoà sẽ hiện và sai rằng đó là nơi duy nhất. Nó cũng không được đặt tốt. “Tương đương hiệu năng” đọc như một đại lượng và nêu ít nhất ba: vòng khứ hồi giao dưới bão

hoà, mã các trình biên dịch phát, và việc bền dưới quá tải. Ba cái đó không đi cùng nhau.

H2 Với bảy lớp được tiêm cùng loại, nó giữ không ngoại lệ. Nó thất bại ở cách đọc lỏng hơn mà cách diễn đạt mời. Nó được chấp nhận cho an toàn bộ nhớ và không tranh chấp dữ liệu, bị bác cho tính đúng đắn tương tranh nói chung, và nó là tuyên bố chỉ về Rust *an toàn*. Trường hợp đường nối cho thấy cùng lớp trở lại bên trong `unsafe`.

H3 Mọi thuế mỗi lần gọi được giới hạn ở 10.91% của cuộc gọi báo hiệu đó, đó là chỗ dựa từ một giới hạn trên một chi phí mà phép đo có thể đã bỏ sót, không từ thất bại phát hiện một chi phí. Tuyên bố kèm theo, rằng chi phí thực là thiết kế và đào tạo, được chống đỡ về cấu trúc chứ không về thống kê.

6.4 Hướng dẫn kỹ thuật cho áp dụng tăng dần

Các hướng dẫn là dạng thực tiễn của các phát hiện, và mỗi cái mang giới hạn của phát hiện đứng sau. Chọn thành phần đầu vì giao diện hẹp, không vì kích thước. Thiết kế lại một giao diện giàu là rủi ro lịch. Bản thân biên không phải. Giữ ABI C trong crate riêng và lõi không `unsafe`. Điều đó giữ mã `unsafe` khỏi các chuỗi phụ thuộc mà nó nếu không lan qua [56]. Để ngân sách rà soát tương tranh ở chỗ cũ. Ưa một cấu trúc không chia sẻ ở ngôn ngữ nào. Kết quả tương tranh mạnh nhất ở đây thuộc về thiết kế chứ không thuộc trình biên dịch nào. Lập kế hoạch đường cong học tập như một chi phí đào tạo và đo nó. Đo thành phần cũng như vòng khứ hồi. Quét qua điểm dịch vụ ngừng chịu được. Đó là hướng dẫn nghiên cứu này muốn đã theo từ đầu nhất.

6.5 Nơi nghiên cứu này đứng, và những gì nó không chống đỡ được

Các nghiên cứu gần nhất với nghiên cứu này là năm luận văn do nhà cung cấp lưu trữ của Mục 2.3. Phán quyết của chúng mâu thuẫn nhau theo mỗi chiều. Bài báo dấu ngược trên bộ nhớ đỉnh hoà với nghiên cứu này khi các nhánh được đọc như hai thời gian chạy tương tranh chứ không phải hai ngôn ngữ. Ba mối đe dọa vẫn mang sức, và cái đầu là một tác giả viết cả hai bản. Cùng dạng thất bại đã đảo một phán quyết đã công bố trong ngành này: tương đương sau một rà soát mã rồi Rust kém hơn 85–91% sau tối ưu thêm [33]. Bộ tương đương chặn hành vi chứ không chặn hiệu năng. Dụng

cụ đầu-cuối đo thiết kế trung thành và ngôn ngữ hầu như không đo. Phân kỳ quá tải được tái hiện ở đây với cơ chế được thu hẹp chứ không đóng, và không nghiên cứu đã bình duyệt nào báo cùng sự đảo.

6.6 Suy nghĩ đạo đức và nghề nghiệp

Nghiên cứu không xử lý dữ liệu cá nhân, thuê bao, hay mật của nhà khai thác. Mọi lưu lượng đều tổng hợp và phân hệ được mô hình hoá theo mã đang chạy chứ không sao chép từ đó, nên các hiện vật có thể được phát hành công khai. Trách nhiệm thứ nhất là không tuyên bố quá. Một nhà cung cấp có thể đọc kết quả RQ2 như giấy phép mô tả một thành phần đã di trú là “an toàn”, và các trường hợp biên là lý do cách đọc đó sai. Một mặt điều khiển khoá chết là một sự cố. Một cập nhật bộ đếm mất trong im lặng có thể làm sai một quyết định di động. Không gì được chứng minh ở đây ngăn một trong hai. Trách nhiệm thứ hai là bằng chứng đi xa đến đâu. Các số độ trễ đuôi từ một máy đơn là cơ sở hẹp hơn vẻ ngoài cho một cam kết sản xuất. Đó là lý do chiến dịch máy ảo trước được báo cáo chứ không bị loại.

7 Kết luận và hướng phát triển

Luận văn so sánh C++ hiện đại và Rust. Đối tượng là một dịch vụ Quản lý Ngữ cảnh UE mô hình hoá theo một thành phần gNB-CU-CP thực của Nokia. Nó nhìn bốn chiều: khả năng dự đoán độ trễ đuôi, ngăn lỗi lúc biên dịch, chi phí mô hình an toàn, và chi phí thêm Rust từng phần. Cả hai bản được xây trên một đặc tả cố định và được đưa đi theo cùng các đường mã. Đó là điều cho phép tách một hiệu ứng ngôn ngữ khỏi một khác biệt thiết kế hay kỹ năng.

Về an toàn, khác biệt là khác biệt về loại và nó có một biên rõ. Rust an toàn từ chối cả bảy lớp an toàn bộ nhớ và tranh chấp dữ liệu được tiêm vào tại thời điểm biên dịch. Các mối nguy mà C++ chỉ có thể hy vọng bắt dưới một kiểm thử bộ dò lỗi trở thành những chương trình không thể viết. Cả bảy lớp biên đều biên dịch và hành xử sai trong Rust an toàn như trong C++. Trên hai lớp trong số đó công cụ C++ báo những gì công cụ Rust không báo. Nêu chính xác, Rust loại tranh chấp dữ liệu và lỗi aliasing. Nó không loại điều kiện đua, lỗi thứ tự khoá, rò rỉ, số học quấn hay phạm vi thiệt hại của một luồng xử lý thất bại, và nó không loại gì bên trong một khối `unsafe`. Nơi các lỗi biên bị bắt, chúng bị bắt lúc kiểm thử. LeakSanitizer và UndefinedBehaviorSanitizer báo mà không cần viết lại. Hai bộ kiểm tra mô hình chỉ báo các trường hợp tương tranh sau khi lỗi được viết lại vào nguyên thủy của chúng.

Về di trú, chi phí bị chiếm bởi thiết kế API và đường cong học tập, không bởi bất kỳ thuế lúc chạy nào. Một lõi Rust không đổi có thể được điều khiển từ C++ qua một đường nối mỏng. Chi phí trên một cuộc gọi báo hiệu thực được giới hạn ở 10.91% của cuộc gọi đó. Toàn bộ `unsafe` nằm ở đường nối, và lõi được xác nhận không có nó. Phần lớn chi phí thêm của lần build Rust là biên dịch phụ thuộc, không phải công việc của chính trình biên dịch. Khoảng cách kích thước nhị phân là một chính sách liên kết. Thứ tự phụ thuộc chính sách khớp nào được chọn. Nơi C++ dẫn là vòng sửa-biên dịch, và nó chỉ dẫn với các sửa nằm trong một đơn vị dịch.

Về hiệu năng, bằng chứng chia trước theo cái đang được đo. Một vòng khứ hồi đầu-cuối ở đây là 0.250% công việc ngôn ngữ. Phần còn lại là bàn giao của hệ điều hành. Đồng hồ đó vì vậy không giới hạn gì về trình biên dịch nào. Cùng lõi đo riêng nhanh hơn 1.6× ở Rust. Tại tải đại diện 56 kops/s, kiểm định ghép cặp phân giải 1.3 μs nghiêng về Rust, quá nhỏ để một triển khai bận tâm ở trung vị. Tại p99.99 biến thể Rust chậm nhất nằm 50 μs dưới biến thể C++ nhanh nhất. Các nhánh Rust rồi giữ tốc độ đưa

vào đến 448 kops/s đối với 392. Khi cả bốn đã sụp, C++ duy trì 1.7× thông lượng của Rust, cái tốn 1.8× CPU trên mỗi thao tác nó hoàn thành. Về bộ nhớ, bản build Rust mang thêm vài MiB dấu chân cố định dưới cả hai bộ cấp phát, cùng chiều và cùng cỡ trên cả ba máy được đo. Lựa chọn bộ cấp phát tốn khoảng chừng đó trên cả hai máy vật lý, và nhiều hơn một bậc trên máy ảo nghiên cứu này dùng trước.

Gộp lại, chúng ủng hộ một tuyên bố hẹp. Nó dành cho các dịch vụ mặt điều khiển có trạng thái, tương tranh, phân vùng được thuộc loại này. Ở các tải mà dịch vụ vẫn đáp tốc độ đưa vào, lựa chọn là những lớp lỗi nào một nhóm muốn làm không thể viết. Nửa kia là liệu nhóm sẽ trả chi phí thiết kế và đào tạo để có chúng. Qua điểm đó nó cũng là một quyết định tốc độ. Điều luận văn thêm là một dụng cụ đo, không phải một phán quyết. Dụng cụ đó là đặc tả cố định và cặp được kiểm thử tương đương. Nó cũng là một khung đo thời gian đi qua điểm gối bão hoà. Một phép tách vòng khứ hồi nói ngôn ngữ có thể chịu bao nhiêu phần của những gì khung đo. Một danh mục lỗi vẽ nơi bảo đảm dừng.

Công việc tương lai trả lời các khoảng trống từng cái một. Việc đầu không cần máy mới: đổi chính sách transparent hugepage trên một máy chủ với mọi thứ khác giữ cố định. Cả ba máy đồng ý rằng bản build Rust mang dấu chân lớn hơn. Chúng không đồng ý về việc đó so với chi phí bộ cấp phát thế nào. Máy ảo đặt nó cao hơn một bậc, và không máy vật lý nào làm vậy. Chính sách hugepage là ứng viên nghiên cứu này có thể nêu tên nhưng không thử được, vì nó khác giữa các máy và chỉ nhánh bộ cấp phát hệ thống thừa hưởng nó. Ngoài ra, phát lại các vết sản xuất đã ẩn danh sẽ kiểm hỗn hợp thao tác. Khung cũng có thể nhận thêm phân hệ mặt điều khiển hoặc đường tầng 2 và tầng vật lý (PHY) nặng dữ liệu. Một phân hệ có trạng thái khả biến dùng chung thực sự sẽ tập hai ngôn ngữ khác với một thiết kế không chia sẻ. Thử các thời gian chạy async Rust đối với mô hình một luồng mỗi phân mảnh sẽ tách thời gian chạy khỏi ngôn ngữ trong kết quả đuôi [32]. Đo chi phí cô lập trong tiến trình của hai heap sẽ định giá việc bảo vệ một lõi không `unsafe` đáng bao nhiêu [60, 61]. Một nghiên cứu tình huống dài hơn theo dõi số lỗi và công sức rà soát trong khi thêm Rust từng phần là điều sẽ chốt chi phí con người mà nghiên cứu này chỉ có thể nêu tên.

Tuyên bố về việc dùng công cụ AI

Công cụ. Claude Opus 5 của Anthropic và Cursor Grok 4.6 là các hệ thống AI sinh được dùng khi soạn luận văn này. Cả hai chạy qua môi trường phát triển Cursor, từ tháng 6 2026 đến khi viết luận văn. Chính sách Đại học Helsinki cấm nêu một mô hình ngôn ngữ như một tác giả. Không mô hình nào được trích như một nguồn, đó là lựa chọn của tác giả.

Phạm vi hỗ trợ. Các mô hình soạn thảo và sửa văn xuôi của luận văn, và đề xuất thứ tự các lập luận. Chúng viết và tái cấu trúc cả hai bản dịch vụ Quản lý Ngữ cảnh UE. Chúng cũng viết bộ sinh tải, danh mục tiêm lỗi, và nguyên mẫu FFI. Phân tích đứng sau các khoảng bootstrap và các kiểm định hoán vị ghép cặp cũng là công việc của chúng, cũng như các nguồn hình, build và LaTeX. Tóm tắt là bài kiểm tra trưởng thành của chương trình này. Chính sách đại học không bao giờ cho phép một mô hình ngôn ngữ trong một bài như vậy. Tóm tắt do tác giả viết không có hỗ trợ.

Những gì các mô hình không quyết. Bài toán nghiên cứu, bốn câu hỏi nghiên cứu, ba giả thuyết, và kiểm soát chính của nghiên cứu là của tác giả. Kiểm soát đó là một đặc tả trung lập ngôn ngữ đã cố định và một mô hình tương tranh, nên ngôn ngữ vẫn là khoảng cách có kế hoạch duy nhất giữa hai bản. Tác giả cũng chọn mang các lớp lỗi từ ngoài bảo đảm lúc biên dịch của Rust, điều giữ RQ2 có thể bác bỏ. Mọi phép đo do tác giả chạy, trên các máy mà mỗi kết quả nêu. Các đại lượng luận văn trích tới trang như macro được xây lại từ nhật ký thô. Cách đọc cũng là của tác giả.

Đầu ra được kiểm thế nào. Mọi đầu ra của mô hình, văn xuôi và mã, đều được tác giả đọc trước khi vào luận văn. Mọi mục thư mục được đối chiếu với Crossref, DataCite, hoặc cơ quan phát hành. Một cuộc kiểm rồi chấm mức mỗi tuyên bố truy được. Bậc thứ nhất là chính trang hoặc mệnh đề. Bậc thứ hai là hồ sơ nhà xuất bản nộp mang tuyên bố. Bậc thứ ba là siêu dữ liệu đã xác nhận sau tường phí. Một mục mà đoạn chống đỡ chưa bao giờ được mở thì được liệt kê, không được hấp thụ. Cuộc kiểm tìm bốn đoạn phóng đại hoặc đặt sai nguồn. Một đoạn là một trích dẫn vắng trong công trình được gán. Cả bốn đã được sửa. Mã được kiểm đối với một bộ tuân thủ mà bản ghi được diff từng dòng. Nó cũng được kiểm đối với một danh mục lỗi chốt mỗi kết cục kỳ vọng trước khi lỗi được tiêm.

Trách nhiệm. Tác giả đã rà toàn bộ luận văn này và chịu trách nhiệm về tất cả. Nơi nó sai, lỗi là của tác giả.

Tài liệu tham khảo

[1] 3GPP. *NG-RAN; Architecture description*. Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip.

[2] 3GPP. *NR; NR and NG-RAN Overall description; Stage 2*. Technical Specification (TS) 38.300. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.300/38300-h50.zip.

[3] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia. "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges". In: *IEEE Communications Surveys & Tutorials* 25.2 (2023), pp. 1376–1411. DOI: 10.1109/COMST.2023.3239220. URL: https://doi.org/10.1109/COMST.2023.3239220.

[4] 3GPP. *NR; Radio Resource Control (RRC); Protocol specification*. Technical Specification (TS) 38.331. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), July 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.331/38331-h50.zip.

[5] 3GPP. *Study on scenarios and requirements for next generation access technologies*. Technical Report (TR) 38.913. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms. 3rd Generation Partnership Project (3GPP), Apr. 2024. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/38913-i00.zip.

[6] J. Dean and L. A. Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794.

[7] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan. "Attack of the Killer Microseconds". In: *Communications of the ACM* 60.4 (2017), pp. 48–54. DOI: 10.1145/3015146. URL: https://doi.org/10.1145/3015146.

[8] M. Miller. *Trends, challenge, and shifts in software vulnerability mitigation.* BlueHat IL, Microsoft Security Response Center (MSRC). Approximately 70% of Microsoft CVEs are memory-safety issues. 2019. URL: https://github.com/microsoft/MSRC-Security-Research/tree/master/presentations/2019_02_BlueHatIL.

[9] J. Vander Stoep. *Memory Safe Languages in Android 13.* Google Security Blog. Dec. 2022. URL: https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html.

[10] S. Klabnik and C. Nichols. *The Rust Programming Language.* 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6. URL: https://openlibrary.org/isbn/9781718503106.

[11] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "RustBelt: Securing the Foundations of the Rust Programming Language". In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018), 66:1–66:34. DOI: 10.1145/3158154. URL: https://doi.org/10.1145/3158154.

[12] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk. "System Programming in Rust: Beyond Safety". In: *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS).* 2017, pp. 156–161. DOI: 10.1145/3102980.3103006. URL: https://doi.org/10.1145/3102980.3103006.

[13] M. Polese, N. Mohamadi, S. D'Oro, L. Bonati, and T. Melodia. "Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G". In: *IEEE Communications Magazine* 64.7 (2026), pp. 90–95. DOI: 10.1109/MCOM.001.2500438. URL: https://doi.org/10.1109/MCOM.001.2500438.

[14] ITU-R. *Framework and overall objectives of the future development of IMT for 2030 and beyond.* Recommendation ITU-R M.2160-0. Approved 13 November 2023. Defines the IMT-2030 ("6G") framework and its development timeline. International Telecommunication Union, Radiocommunication Sector, Nov. 2023. URL: https://www.itu.int/rec/R-REC-M.2160/en.

[15] AI-RAN Alliance. *AI-RAN Platform and Infrastructure Orchestrator.* AI-RAN Alliance, Working Group 2 (AI-and-RAN). Version 10-rc1, clause 2.2. Defines AI-and-RAN as one common IT infrastructure accommodating both AI and RAN workloads, which the infrastructure is required to run concurrently, and notes that the two kinds of workload may have different service-level agreements. Contributors from Ericsson, Nokia, Nvidia, Red Hat, Samsung, SoftBank, zTouch Networks

and Northeastern University. Accessed August 2026. Feb. 2026. URL: https://ai-ran.org/documents/AI-RAN-Platform-Infrastructure-Orchestrator.pdf.

[16] J. Vander Stoep. *Rust in Android: Move Fast and Fix Things*. Google Security Blog. Nov. 2025. URL: https://blog.google/security/rust-in-android-move-fast-fix-things/.

[17] A. Rebert and C. Kern. *Secure by Design: Google's Perspective on Memory Safety*. Google Security Engineering, Technical Report. 2024. URL: https://research.google/pubs/secure-by-design-googles-perspective-on-memory-safety/.

[18] R. N. M. Watson, J. Baldwin, D. Chisnall, T. Chen, J. Clarke, B. Davis, N. W. Filardo, B. Gutstein, G. Jenkinson, B. Laurie, A. Mazzinghi, S. W. Moore, P. G. Neumann, H. Okhravi, A. Richardson, A. Rebert, P. Sewell, L. Tratt, M. Vijayaraghavan, H. Vincent, and K. Witaszczyk. "It Is Time to Standardize Principles and Practices for Software Memory Safety". In: *Communications of the ACM* 68.2 (2025), pp. 40–45. DOI: 10.1145/3708553. URL: https://doi.org/10.1145/3708553.

[19] Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously*. Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps.

[20] Office of the National Cyber Director. *Back to the Building Blocks: A Path Toward Secure and Measurable Software*. Technical Report. Washington, DC: The White House, Office of the National Cyber Director (ONCD), Feb. 2024. URL: https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf.

[21] European Parliament and Council of the European Union. *Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)*. Official Journal of the European Union, OJ L, 2024/2847. Nov. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[22] Dell'Oro Group. *RAN Mostly Stable in 3Q 2025*. Dell'Oro Group, RAN Quarterly Report summary. The five largest RAN suppliers by worldwide revenue (Huawei, Ericsson, Nokia, ZTE, Samsung) accounted for 96% of the 1Q25–3Q25

RAN market, up from 95% in 2024. Accessed August 2026. Nov. 2025. URL: https://www.delloro.com/news/ran-mostly-stable-in-3q-2025/.

[23] Dell'Oro Group. *RAN Market Stabilized in 2025*. Dell'Oro Group, RAN Quarterly Report summary. Full-year 2025 preliminary findings: supplier rankings unchanged, and overall market concentration measured by the Herfindahl–Hirschman Index reached a ten-year high. Accessed August 2026. Feb. 2026. URL: https://www.delloro.com/news/ran-market-stabilized-in-2025/.

[24] J. Almodovar and A. Sultan. *The SA1 road to 6G*. 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work. ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road.

[25] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. "Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs". In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2020, pp. 763–779. DOI: 10.1145/3385412.3386036. URL: https://doi.org/10.1145/3385412.3386036.

[26] A. Meneely, A. Green, T. Jaafari, M. Fluet, and B. Keller. ""Just Use Rust": A Best-Case Historical Study of Open Source Vulnerabilities in C". In: *Proceedings of the 3rd IEEE/ACM International Workshop on Software Vulnerability Management (SVM)*. 2025, pp. 25–32. DOI: 10.1109/SVM66695.2025.00008. URL: https://doi.org/10.1109/SVM66695.2025.00008.

[27] 3GPP. *Service requirements for the 5G system; Stage 1*. Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip.

[28] 3GPP. *NG-RAN; E1 Application Protocol (E1AP)*. Technical Specification (TS) 38.463. V17.0.0, Release 17. 3rd Generation Partnership Project (3GPP), Apr. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.463/38463-h00.zip.

[29] R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Sci-

ence, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307.

[30] V. Olsson. "Rust Programming Language as an Alternative to C for RAN Management Applications". Carried out at Ericsson AB. Master's thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822.

[31] J. Olsson and I. Stenström. "Rust Async for Thread-per-Core Packet Processing". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029.

[32] A. Arvidsson and M. Kaller. "Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746.

[33] C. Emberg and A. Wadsten. "Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of ASN.1 Messages". Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552.

[34] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva. "Energy Efficiency Across Programming Languages: How Do Energy, Time, and Memory Relate?" In: *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*. 2017, pp. 256–267. DOI: 10.1145/3136014.3136031. URL: https://doi.org/10.1145/3136014.3136031.

[35] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. "The Case for Writing Network Drivers in High-Level Programming Languages". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–13. DOI: 10.1109/ANCS.2019.8901892. URL: https://doi.org/10.1109/ANCS.2019.8901892.

[36] F. Suchert and J. Castrillon. "STAMP-Rust: Language and Performance Comparison to C on Transactional Benchmarks". In: *Benchmarking, Measuring, and Optimizing (Bench 2022)*. Vol. 13852. Lecture Notes in Computer Science. Springer, 2023, pp. 160–175. DOI: 10.1007/978-3-031-31180-2_10. URL: https://doi.org/10.1007/978-3-031-31180-2_10.

[37] R. Gjomemo, P. H. Phung, E. Ballou, K. S. Namjoshi, V. N. Venkatakrishnan, and L. Zuck. "Leveraging Static Analysis Tools for Improving Usability of Memory Error Sanitization Compilers". In: *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. Two of the six authors are affiliated with Nokia. 2016, pp. 323–334. DOI: 10.1109/QRS.2016.44. URL: https://doi.org/10.1109/QRS.2016.44.

[38] B. Schroeder, A. Wierman, and M. Harchol-Balter. "Open Versus Closed: A Cautionary Tale". In: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*. 2006. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale.

[39] R. Natella, D. Cotroneo, and H. S. Madeira. "Assessing Dependability with Software Fault Injection: A Survey". In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425.

[40] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. "Detecting Cross-Language Memory Management Issues in Rust". In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33.

[41] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. ISBN: 978-0-412-04231-7.

[42] S. Holm. "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6 (1979), pp. 65–70. URL: https://www.jstor.org/stable/4615733.

[43] P. Enberg, A. Rao, and S. Tarkoma. "The Impact of Thread-per-Core Architecture on Application Tail Latency". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874.

[44] G. Tene. *HdrHistogram: A High Dynamic Range Histogram*. Project documentation and reference implementation. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: https://github.com/HdrHistogram/HdrHistogram.

[45] T. Yin. *Lizard: An extensible cyclomatic complexity analyzer for many programming languages*. Version 1.23.0. Accessed August 2026. 2026. URL: https://github.com/terryyin/lizard.

[46] A. Georges, D. Buytaert, and L. Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033.

[47] X. Zhou, V. Leis, X. Yu, and M. Stonebraker. "OLTP Through the Looking Glass 16 Years Later: Communication Is the New Bottleneck". In: *15th Annual Conference on Innovative Data Systems Research (CIDR 2025)*. 2025. URL: https://www.vldb.org/cidrdb/papers/2025/p17-zhou.pdf.

[48] J. Hughes and L. Tratt. "Garbage Collection for Rust: The Finalizer Frontier". In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA2 (2025), pp. 3588–3614. DOI: 10.1145/3763179. URL: https://doi.org/10.1145/3763179.

[49] M. Cui, H. Xu, H. Tian, and Y. Zhou. "rCanary: Detecting Memory Leaks Across Semi-Automated Memory Management Boundary in Rust". In: *IEEE Transactions on Software Engineering* 50.9 (2024), pp. 2472–2484. DOI: 10.1109/TSE.2024.3443624. URL: https://doi.org/10.1109/TSE.2024.3443624.

[50] J. Bornholt, R. Joshi, V. Astrauskas, B. Cully, B. Kragl, S. Markle, K. Sauri, D. Schleit, G. Slatton, S. Tasiran, J. Van Geffen, and A. Warfield. "Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3". In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 836–850. DOI: 10.1145/3477132.3483540. URL: https://doi.org/10.1145/3477132.3483540.

[51] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. "Towards Understanding the Runtime Performance of Rust". In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2022, 140:1–140:6.

DOI: 10.1145/3551349.3559494. URL: https://doi.org/10.1145/3551349.3559494.

[52] S. Harris, S. Cooksey, M. Vollmer, and M. Batty. "Rust for Morello: Always-On Memory Safety, Even in Unsafe Code (Experience Paper)". In: *Proceedings of the 37th European Conference on Object-Oriented Programming (ECOOP)*. Vol. 263. Leibniz International Proceedings in Informatics (LIPIcs). 2023, 39:1–39:27. DOI: 10.4230/LIPIcs.ECOOP.2023.39. URL: https://doi.org/10.4230/LIPIcs.ECOOP.2023.39.

[53] Y. Zhang, K. Zhang, G. Liu, Y. Cai, and S. Qin. "Two Birds One Stone: Effective Static Detection of Resource and Communication Deadlocks in Rust Programs". In: *Automated Software Engineering* 33.2, 65 (2026). Extends the preprint arXiv:2401.01114. DOI: 10.1007/s10515-026-00614-z. URL: https://doi.org/10.1007/s10515-026-00614-z.

[54] S. Lu, S. Park, E. Seo, and Y. Zhou. "Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics". In: *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2008, pp. 329–339. DOI: 10.1145/1346281.1346323. URL: https://doi.org/10.1145/1346281.1346323.

[55] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. "Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs". In: *ACM Transactions on Software Engineering and Methodology* 31.1 (2022), 3:1–3:25. DOI: 10.1145/3466642. URL: https://doi.org/10.1145/3466642.

[56] A. N. Evans, B. Campbell, and M. L. Soffa. "Is Rust Used Safely by Software Developers?" In: *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 2020, pp. 246–257. DOI: 10.1145/3377811.3380413. URL: https://doi.org/10.1145/3377811.3380413.

[57] A. Sundvisson and S. Hofer. "User-Space Device Drivers on Embedded Linux: A Comparative Case Study of C and Rust". Bachelor's thesis. Blekinge Institute of Technology, Department of Software Engineering, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-29775.

[58] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. "Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study". In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[59] W. Crichton and S. Krishnamurthi. "Profiling Programming Language Learning". In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA1 (2024), pp. 29–54. DOI: 10.1145/3649812. URL: https://doi.org/10.1145/3649812.

[60] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. "Keeping Safe Rust Safe with Galeed". In: *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*. 2021, pp. 824–836. DOI: 10.1145/3485832.3485903. URL: https://doi.org/10.1145/3485832.3485903.

[61] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz. "PKRU-Safe: Automatically Locking Down the Heap Between Safe and Unsafe Languages". In: *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*. 2022, pp. 132–148. DOI: 10.1145/3492321.3519582. URL: https://doi.org/10.1145/3492321.3519582.